

基于空间几何优化的烟幕干扰弹投放策略

摘要

本文针对烟幕干扰弹的投放策略优化问题,运用**运动学定律及空间几何位置关系**进行机理分析,建立多约束条件优化模型,基于**二分查找、蒙特卡洛**进行计算机求解。在不同无人机数量、烟幕干扰弹数量以及导弹数量的多种情形下,求解出最优投放策略。

针对问题一,在给定单机飞行速度、方向,烟幕干扰弹投放及起爆时间的条件下,本文首先对问题场景建立空间直角坐标系,并建立导弹运动模型、无人机运动模型以及烟幕干扰弹运动模型以描述三者物理动态运动。其次,本文定义“**完全遮蔽模型**”以计算有效遮蔽时长。然后通过真目标上取点进行射线投射以模拟导弹视线。同时将 20 秒的有效遮蔽时段进行离散化处理,采用时序差分方法逐时间步长判断该时间片内是否满足完全遮蔽条件,累积得到该条件下**有效遮蔽时长为 1.39s**。

针对问题二,对于优化单无人机单枚烟幕干扰弹对单枚导弹的投放策略,在建立以有效遮蔽时长为目标函数的优化模型后,若直接采用遗传算法等启发式搜索策略,往往因解空间维度高而计算效率低下,且容易陷入局部最优。因此本文基于运动学规律和空间几何约束,提出“**圆柱轨迹模型**”,通过解析相对运动轨迹显著压缩了搜索空间,排除了大量不可行与次优区域。在此基础上进行精细搜索,可在保证解的质量的同时显著提高计算效率。得出**有效遮蔽时长为 4.60s**的最优策略。

针对问题三,对于单机连续投放三弹以遮蔽单枚导弹的协同优化问题,为避免高维决策空间下的数值搜索,本文再次从运动学与几何关系入手,建立“**最优遮蔽模型**”,计算出**最佳起爆位置线**与**云团最优分布线**,将原问题转化为对烟幕干扰弹最佳起爆位置的求解。模型同时考虑了多弹投放间隔与最小化遮蔽重叠时间的约束。然后采用二分查找高效确定各弹投放点,并反向推演无人机的航速航向及投弹起爆时序。得出**有效遮蔽时长为 7.51s**的最优策略。

针对问题四,对于三架无人机各投一弹协同遮蔽单枚导弹的优化策略,本文延续采用“圆柱轨迹模型”,重点聚焦**多弹时序协同机制**:优化投弹与起爆时序,尽可能避免遮蔽时段重叠,以实现时序无缝衔接持续干扰。得出**有效遮蔽时长为 11.78s**的最优策略。

针对问题五,对于优化五架无人机至多投放三弹实现对三枚导弹的最优遮蔽,本文沿用“最佳遮蔽模型”的思路,推导出每架无人机针对每枚导弹的最佳起爆位置及云团最优位置线的**通解表达式**。接着使用问题四中的时序协同与遮蔽重叠约束,对通解结果进行约束优化与可行性校验,最终构建融合空间布局优化与时间协同调度的多目标遮蔽策略。得出对三枚导弹**平均有效遮蔽时长为 11.98s**的最优策略。

最后,我们对模型进行灵敏度分析和误差分析,并评估模型的优缺点。

关键字: 烟幕干扰弹投放策略 空间几何优化 多约束优化 蒙特卡洛 二分查找

一、问题重述

1.1 问题背景

烟幕干扰弹是一种通过释放气溶胶微粒来干扰敌方光电探测和制导系统的军事设备，具有显著的防御功能。烟幕干扰弹在历史上已经多次被应用于实战中，现代军队如中国火箭军已将其发展为系统化装备，由此可见其显著的战术价值和应用场景。因此提高烟幕弹的干扰效能对于更好地应对现代战争的挑战具有重要性。

现结合无人机飞行实况讨论烟幕干扰弹投放策略：无人机投放两枚烟幕弹之间的时间间隔至少为 1 秒；烟幕干扰弹爆炸后形成的球状云团以 $3m/s$ 速度匀速下沉，并认为云团中心 10m 范围内可以在 20 秒内有效遮蔽目标；导弹始终以 $300m/s$ 的速度向一个半径 7m，高 10m 的圆柱形假目标飞行；已知在以假目标为坐标原点的三维坐标系下真实目标、三枚导弹以及 5 架无人机的位置坐标。

为了探究如何使得投放烟幕弹的策略更加有效，我们希望规划每架无人机的飞行方、飞行速度，以及烟幕干扰弹投放点和起爆点，使烟幕干扰弹对真实目标的有效遮蔽时间尽可能长。这是一个多目标，多变量，多约束的复杂动态规划问题。

1.2 问题要求

问题 1 位于 (17800, 0, 1800) 处的无人机 FY1 以 $120m/s$ 速度在 1.5s 后投放一枚烟幕干扰弹，并在投放后 3.6s 后起爆烟幕干扰弹，求解这种情况下烟幕干扰弹对导弹 M1 的有效遮蔽时长。

问题 2 仍然考虑无人机 FY1 投放一枚烟幕干扰弹以阻碍导弹 M1 的情况，探究 FY1 的飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点四个参数如何取值，使得干扰弹对目标的有效遮蔽时长最大。

问题 3 考虑无人机 FY1 投放三枚烟幕干扰弹对导弹 M1 进行干扰，探究无人机飞行速度、飞行方向以及每一枚烟幕弹的投放坐标以及起爆坐标，使得有效遮蔽时长最大。

问题 4 考虑三架无人机 FY1, FY2, FY3 分别投放一枚烟幕干扰弹干扰导弹 M1，探究每架无人机飞行速度、飞行方向以及烟幕干扰弹的投放坐标以及起爆坐标，使得有效遮蔽时长最大。

问题 5 考虑五架无人机 FY1, FY2, FY3, FY4, FY5 分别投放至多三枚烟幕干扰弹干扰导弹 M1, M2, M3，探究每架无人机飞行速度、飞行方向以及每枚烟幕弹的投放坐标以及起爆坐标，使得有效遮蔽时长最大。

二、问题分析

2.1 问题一分析

问题一要求在给定无人机 FY1 飞行速度，飞行方向，烟幕干扰弹投弹时间以及起爆时间的条件下，计算该单枚烟幕干扰弹对单个导弹的有效遮蔽时长。在明确导弹，无人机以及烟幕干扰弹的运动规律后，建立三维空间坐标系以及三者的三维运动学模型。接着明确有效遮蔽时长的定义并建立了完全遮蔽模型。然后通过蒙特卡洛取点进行射线投射以模拟导弹视线，采用时序差分将 20s 的有效遮蔽时间窗口微分化，统计判定每个微小时间段内导弹和烟幕弹的动态位置关系是否满足完全遮蔽模型，最终计算出单枚烟幕干扰弹对单枚导弹的有效遮蔽时间。

2.2 问题二分析

问题二要求通过优化无人机的飞行方向、飞行速度、烟幕干扰弹投放点及起爆点等决策变量，以最大化其对导弹 M1 的有效遮蔽时间。在构建以遮蔽时长为目标函数的非线性优化模型后，常规思路通常采用遗传算法、粒子群算法等启发式搜索策略进行暴力搜索。然而此类方法往往计算效率较低，且易陷入局部最优解，难以保证解的全局最优性。

为此，本文基于无人机与导弹的运动学特性与空间几何关系，提出“圆柱轨迹模型”。该模型通过深入分析相对运动轨迹与云团遮蔽的时空约束，有效排除大量不可行及次优的决策空间，将原问题的高维搜索范围压缩至一个有限且结构清晰的区域。在此基础上进行智能优化搜索求解出最优解。

2.3 问题三分析

问题三将问题二扩展为通过优化单架无人机投放三枚烟幕干扰弹的投放策略来实施对导弹 M1 的有效遮蔽。直接采用数值模拟方法搜索整个决策空间计算效率过低，故解题的重点在于结合运动学规律以及几何关系挖掘烟幕干扰弹投放的空间几何特性。

我们把解题的重心放在基于导弹与真目标的视线几何关系建立烟幕干扰弹云团的“最优遮蔽模型”，拟合出最佳起爆线以及云团最佳位置线，将原问题转换成求解烟幕弹的最佳起爆位置。然后基于对最优遮蔽位置的分析，利用二分搜索的算法，找出投放的最佳点。最后反向推导得出无人机的飞行速度、飞行方向、投弹及起爆的时序。求解过程中综合考虑多枚烟幕干扰弹投放时间间隔约束，并加入最小化重叠有效遮蔽时间的约束，以得出三枚烟幕弹的协同投放策略。

2.4 问题四分析

问题四在问题二的基础上要求探讨三架无人机分别投放一枚烟幕干扰弹实现对导弹 M1 的最优干扰策略。本问题延续使用问题二中的“圆柱轨迹模型”，并将解题重心放在研究多枚烟幕干扰弹在时间序列上的衔接机制——优化投弹起爆时序使得三枚烟幕干扰弹形成的遮蔽时段尽可能不产生重叠，并探究三枚烟幕干扰弹各自的最优解与协同之后的全局最优解的关系。

2.5 问题五分析

问题五将研究内容进一步扩展，旨在优化五架无人机协同投放至多三枚烟幕干扰弹，以实现对导弹 M1、M2 与 M3 的最优遮蔽。本文延续问题三所建立的“最佳遮蔽模型”框架，通过解析多机-多弹-多目标的复杂遮蔽关系，推导出每架无人机针对每枚导弹的最佳起爆位置及云团最优分布曲线的通解表达式。在此基础上，引入问题四中的时序协同与遮蔽重叠约束，对通解结果进行约束优化与可行性校验，从而构建出一个兼顾空间与时序的多目标协同遮蔽策略。

三、模型假设

为简化问题，本文做出以下假设：

- 不考虑空气阻力对物体运动的影响，即在无风环境下运动，烟幕干扰弹在被无人机投放后做平抛运动，云团做匀速下沉运动。
- 将无人机，导弹视为质点。
- 所有计算均在统一的笛卡尔坐标系下进行，不考虑地球的球面几何影响。
- 导弹始终能够在无烟幕干扰弹的遮挡下发现目标，即目标位于视距范围内。

四、符号说明

符号	说明
$\mathbf{M}_i(t)$	第 i 枚导弹在 t 时刻的坐标向量
$\mathbf{V}_{Mi}$	第 i 枚导弹的速度向量
$\mathbf{dir}_{Mi}$	第 i 枚导弹的单位方向向量
$\mathbf{FY}_i(t)$	第 i 架无人机在 t 时刻的坐标向量
$\mathbf{V}_{FYi}$	第 i 架无人机的速度向量
$\mathbf{dir}_{FYi}$	第 i 架无人机的单位方向向量
$\mathbf{D}_i(t)$	第 i 个烟幕弹在 t 时刻的坐标向量
$\mathbf{V}_{Di}(t)$	第 i 个烟幕弹在 t 时刻的速度向量
S	所有的烟幕云团中心集合
$\mathbf{S}_i(t)$	第 i 个烟幕云团中心的坐标向量
R	烟幕云团的有效遮蔽半径
$\mathbf{C}$	真目标圆柱体下底面中心坐标向量
C_{surf}	真目标圆柱体的表面
$\mathbf{P}_c$	真目标圆柱体表面某一点的坐标向量
$\mathbf{O}$	假目标坐标向量
g	重力加速度

五、问题一的模型建立与求解

5.1 问题一的模型建立

5.1.1 环境模型

设警戒雷达发现导弹的时刻为 $t = 0$; 建立以假目标为原点 $(0, 0, 0)$ 的空间直角坐标系, xy 平面为水平面, z 轴垂直向上。 $t = 0$ 时刻后, 来袭导弹 M_1, M_2, M_3 分别位于 $(20000, 0, 2000), ((19000, 600, 2100), (18000, -600, 1900))$ 。 5 架无人机的位置分别为 $FY_1(17800, 0, 1800)$ 、 $FY_2(12000, 1400, 1400)$ 、 $FY_3(6000, -3000, 700)$ 、 $FY_4(11000, 2000, 1800)$ 、 $FY_5(13000, -2000, 1300)$, 形成一道前沿拦截屏障, 以分散阵列占据导弹前方, 共同掩护位于假目标后方位于 $(0, 200, 0)$ 处的真实目标。

三维空间位置图以及俯视图如图1和2所示。

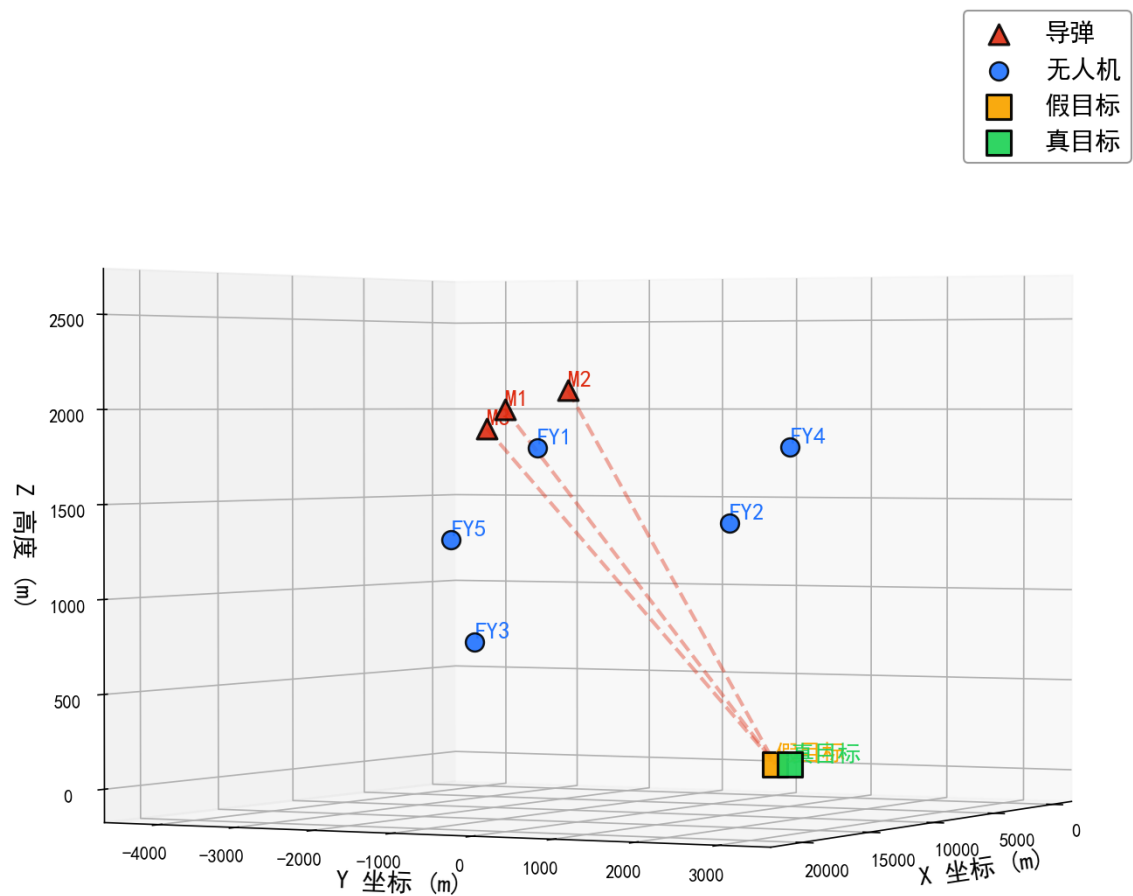


图 1 三维空间建系图

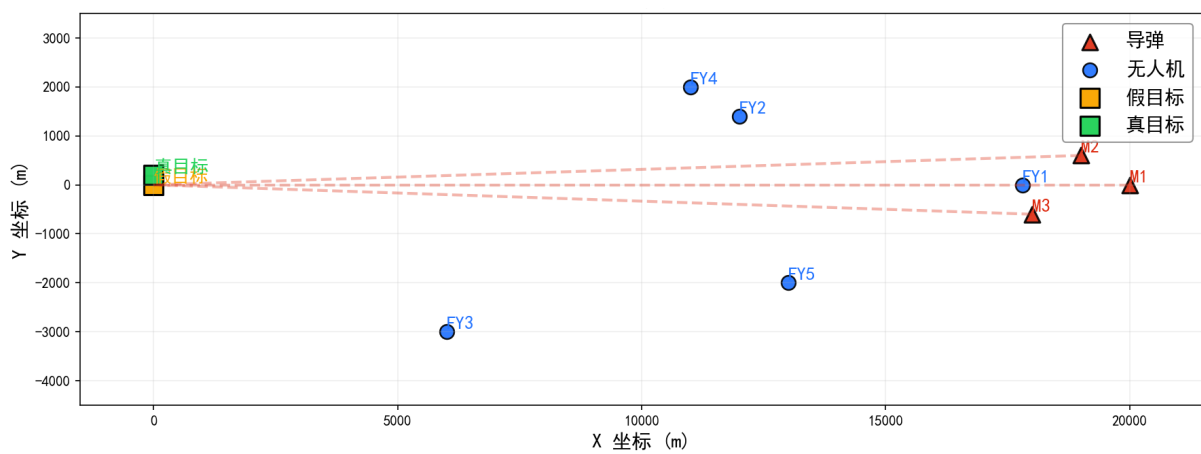


图 2 空间场景俯视图

5.1.2 物体运动模型

1. 导弹运动模型：导弹 M1 的初始位置坐标为：

$$\mathbf{M1}(0) = (20000, 0, 2000)$$

导弹运动方向向量从导弹的位置出发指向假目标点。要求解其单位方向向量，只需要用方向向量除以其模长即可。由于导弹运动方向不变，因此其单位方向向量也不变。单位方向向量计算方式为：

$$\mathbf{dir}_{M1} = \frac{\mathbf{O} - \mathbf{M1}(0)}{\|\mathbf{O} - \mathbf{M1}(0)\|} \quad (1)$$

导弹以 300m/s 的速度匀速直线飞行，导弹的速度向量可以表示为单位方向向量和速度值的乘积，也即：

$$\mathbf{V}_{M1} = 300 \mathbf{dir}_{M1} \quad (2)$$

在此基础上，导弹在任意时刻 t 的位置 $\mathbf{M1}(t)$ 可以用运动学方程计算并表示：

$$\mathbf{M1}(t) = \mathbf{M1}(0) + \mathbf{V}_{M1} t \quad (3)$$

2. 无人机运动模型：无人机 M1 的运动为等高度的匀速直线飞行运动，忽略空气阻力等外力的影响。其初始位置坐标为：

$$\mathbf{FY1}(0) = (17800, 0, 1800)$$

本题中无人机朝向假目标方向飞行，由于无人机是等高度匀速直线飞行，因此可以视为无人机朝向一个等高度的虚拟目标 $\mathbf{O}'$ 飞行，该虚拟目标的 x, y 坐标与假目标相同， z 坐标与无人机相同。即：

$$\mathbf{O}' = (0, 0, 1800)$$

则其单位方向向量表示为：

$$\mathbf{dir}_{FY} = \frac{\mathbf{O}' - \mathbf{FY1}(0)}{\|\mathbf{O}' - \mathbf{FY1}(0)\|} = (-1, 0, 0) \quad (4)$$

本题中无人机的飞行速度固定为 120m/s，因此无人机的速度向量为：

$$\mathbf{V}_{FY1} = 120 \mathbf{dir}_{FY1} \quad (5)$$

进一步将无人机在任意时刻 t 的位置 $\mathbf{FY1}(t)$ 用运动学方程计算并表示：

$$\mathbf{FY1}(t) = \mathbf{FY1}(0) + \mathbf{V}_{FY1} t \quad (6)$$

3. 烟幕干扰弹运动模型：如图3, 假定烟幕干扰弹编号为 1，对于烟幕干扰弹从随无人机飞行到起爆产生遮蔽效果，我们将其飞行轨迹及运动规律划分为未投放、未起爆以及起爆后三个阶段：

第一阶段-未投放：此时无人机携带着烟幕干扰弹做等高度匀速直线飞行运动。烟幕干扰弹的速度等于无人机的飞行速度，烟幕干扰弹的坐标等于无人机的坐标。在 t 时刻，有：

$$\mathbf{V}_{D1}(t) = \mathbf{V}_{FY1} \quad (7)$$

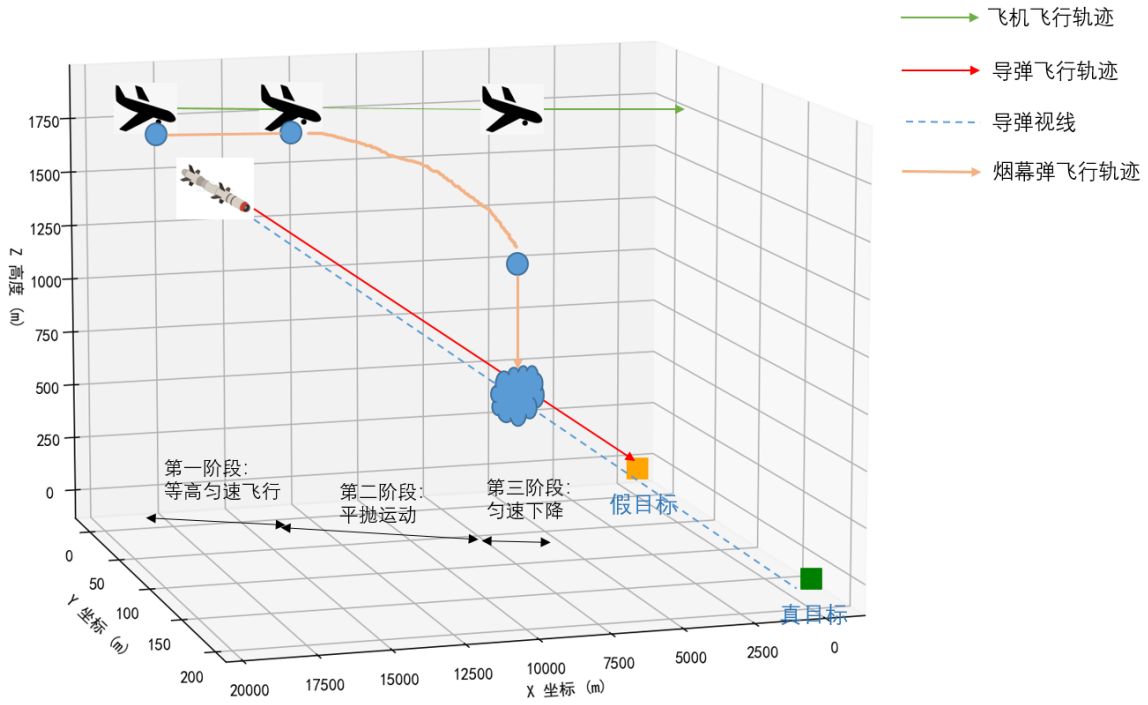


图3 飞行轨迹

$$\mathbf{D}_1(t) = \mathbf{F}\mathbf{Y}_1(t) \quad (8)$$

第二阶段-未起爆：基于我们的模型假设，我们认为烟幕干扰弹被无人机投放后在无风环境中做平抛运动。即水平方向速度仍然保持与无人机飞行速度一致，竖直方向做自由落体运动。假设 $\mathbf{j}$ 为 y 轴正方向，烟幕干扰弹在 t_1 时刻被投放，则在 t 时刻 ($t > t_1$)，有：

$$\begin{cases} \mathbf{V}_{\mathbf{x}_{D1}}(t) = \mathbf{V}_{\mathbf{x}_{FY1}}(t) \\ \mathbf{V}_{\mathbf{y}_{D1}}(t) = -g(t - t_1) \mathbf{j} \end{cases} \quad (9)$$

$$\begin{cases} \mathbf{D}_{\mathbf{x}_1}(t) = \mathbf{F}\mathbf{Y}_1\mathbf{x}(t) \\ \mathbf{D}_{\mathbf{y}_1}(t) = \mathbf{D}_{\mathbf{y}_1}(T_1) - \frac{1}{2}g(t - t_1)^2 \mathbf{j} \end{cases} \quad (10)$$

第三阶段-起爆后：起爆后烟幕干扰弹瞬间形成球状烟幕云团，在采用特定技术下做 $3m/s$ 的匀速下沉运动。假设 $\mathbf{j}$ 为 y 轴正方向，烟幕干扰弹在 t_1 时刻被投放，在 t_2 时刻被引爆，则在 t 时刻 ($t > t_2 > t_1$)，有：

$$\mathbf{V}_{D1}(t) = -3 \mathbf{j} \quad (11)$$

$$\begin{cases} \mathbf{D}_{\mathbf{x}_1}(t) = \mathbf{D}_{\mathbf{x}_1}(0) + \mathbf{V}_{\mathbf{FY1}} t_2 \\ \mathbf{D}_{\mathbf{y}_1}(t) = \mathbf{D}_{\mathbf{y}_1}(t_2) - 3(t - t_2) \mathbf{j} \end{cases} \quad (12)$$

5.1.3 完全遮蔽模型

有效遮蔽时长的定义：我们引入视距的概念——视距指的是人眼或探测器能观察到的最远距离。对于导弹能否发现目标，我们认为目标始终处于导弹的视距范围内。有效遮蔽时长则指烟幕干扰弹从初始时间开始的时间段内，目标被云团完全遮蔽的累计时间。

完全遮蔽目标的定义：我们认为若烟幕云团出现在导弹观测目标的光学红外视线上，即构成对目标的有效遮蔽。具体而言，当导弹通过其探测器试图识别、跟踪和瞄准目标时，云团恰好位于导弹与目标之间的视线路径上，就会阻挡或显著衰减导弹接收到的目标特征信号，从而干扰导弹的目标获取和跟踪能力。

更进一步，完全遮蔽是指烟幕云团不仅遮挡了导弹与目标的直接连线，而是覆盖了导弹视场范围内目标的所有可能观测路径。即云团在三维空间中的投影完全覆盖了目标相对于导弹的可见轮廓。对于圆柱形目标（半径 7m，高 10m），需要确保从导弹视角看去的整个目标截面都被烟幕云团所遮挡。

在我们这种严格定义下，完全遮蔽意味着导弹在其飞行路径上的任何时刻，都无法从任何角度、通过任何缝隙观察到目标的任何部分，从而彻底丧失对真目标的识别和跟踪能力。

对于如何判断云团是否将导弹看到目标的所有视线都遮挡，我们采用基于蒙特卡洛的射线投射方法用于全局可见性分析。蒙特卡洛方法是指从概率分布中随机抽取大量样本，通过统计这些样本的结果来近似解决复杂的确定性或随机性问题。在我们的模型中，我们在半径 7m，高 10m 的圆柱内均匀选取了 N 个点（ N 是一个大于等于 10000 的大整数），这些采样点均匀分布在圆柱体的整个表面（包括顶面、底面和侧面），用以代表导弹可能观测到的目标所有潜在特征点。采样的效果如图4所示（ $N=10000$ ）。

完全遮蔽目标的判断：

判断是否完全遮蔽的流程如下：

1. **视线连接：**从导弹的瞬时位置向每一个目标采样点发射一条虚拟射线，共计生成 N 条射线。这些射线的集合构成了导弹对目标的全视角观测网络。

2. **遮蔽检测：**如图5所示。对每一条射线，我们进行射线-球体求交检测。烟幕云团为一个以起爆点为中心、随时间匀速下沉的半径为 10 米的球体。通过求解射线与球体的交点方程，判断该条视线是否被云团阻挡。若存在交点，则判定该采样点当前被遮蔽；否则该点仍暴露于导弹视野中。要判断云团球体是否遮挡了某条特定的视线，其几何本质是判断一个球体是否与一条视线线段相交。

记导弹 M_i 看向真目标圆柱体上某一点 P_c 的视线线段为 $\mathbf{L}$ ，则

$$\mathbf{L} = \mathbf{P}_c - \mathbf{M}_i(t) \quad (13)$$

一条视线线段 $\mathbf{L}$ 被烟幕云团 S 遮蔽的充要条件是：存在云团球心 $S_i(t)$ 到该视线线

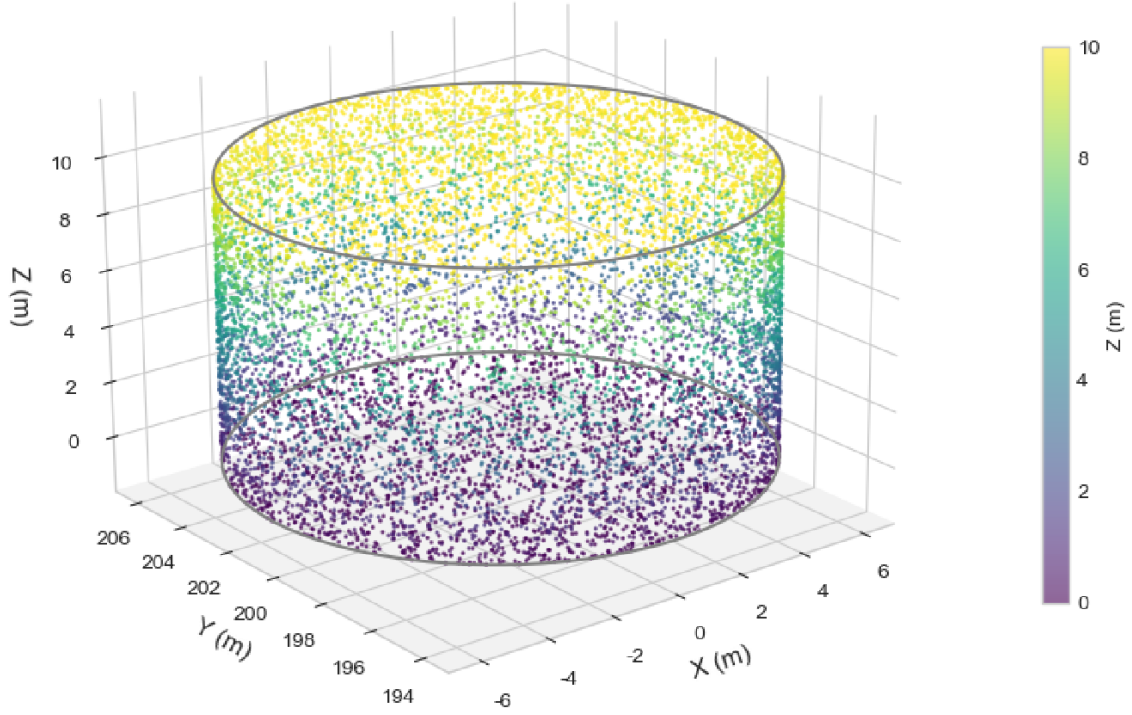


图4 蒙特卡洛取点效果展示

段 $\mathbf{L}$ 的最短距离（即垂线距离或到端点的距离）小于或等于烟幕的半径 R 。记导弹 M_i 与一个烟幕云团中心 S_i 的连线为 $\mathbf{L}'$ ，则

$$\mathbf{L}' = \mathbf{S}_i(t) - \mathbf{M}_i(t) \quad (14)$$

则云团球心 $S_i(t)$ 到该视线线段 $\mathbf{L}$ 的最短距离的数学表达如下：

$$\lambda = \frac{\mathbf{L}' \cdot \mathbf{L}}{\mathbf{L} \cdot \mathbf{L}} \quad (15)$$

$$dist_{min} = \begin{cases} \|\mathbf{L}'\|, & \text{if } \lambda < 0 \\ \|\mathbf{S}_i(t) - \mathbf{P}_c\|, & \text{if } \lambda > 1 \\ \|\mathbf{P}_c - (\mathbf{M}_i(t) + \lambda \mathbf{L})\|, & \text{if } 0 \leq \lambda \leq 1 \end{cases} \quad (16)$$

我们定义完全遮蔽的二值状态函数 $Covered(t)$:

$$Covered(t) = \begin{cases} 1 & \text{if } dist_{min} \leq 10\text{m}, \quad t' = t - (t_{\text{投放}} + t_{\text{起爆}}) \leq 20\text{s} \\ 0 & \text{otherwise} \end{cases} \quad (17)$$

该函数在任意时刻的输出值为 1 表示导弹处于烟幕有效遮蔽范围内，输出值为 0 表示导弹未被遮蔽或遮蔽已失效。

3. 时序差分：在烟幕有效的 20s 时间窗口内，我们将时间划分为多个微小的时间间隔，然后以这个微小的时间步长进行遍历。在每个微小时间片段内，我们都会计算导弹

和烟幕云团在此时的位置，并判断烟幕弹是否完全遮蔽目标。最后我们将所有满足完全遮蔽条件的时间片段累加起来，得到总的有效遮蔽时长。

4. **统计判定：**统计 N 条射线中被成功阻挡的射线数量 N_{occluded} ——即满足 $\text{Covered}(t)=1$ 的射线。我们定义瞬时遮蔽率为：

$$\eta = \frac{N_{\text{occluded}}}{N} \quad (18)$$

当 $\eta = 1$ （即 100% 的采样射线被阻挡）时，我们认为在当前时刻，目标达到了完全遮蔽。在实际应用中，可设定一个阈值（如 $\eta \geq 0.999$ ）来定义有效遮蔽，以容忍极少数因采样或计算精度带来的偏差。

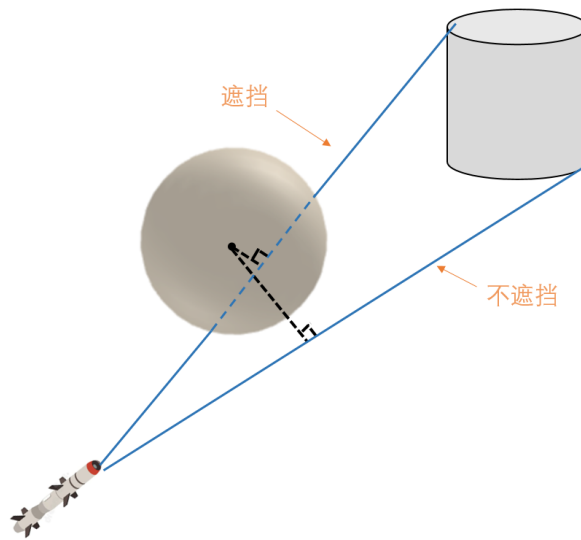


图 5 视线遮挡示意图

5.2 问题一的模型求解

按照题目描述将初始参数定义好——导弹初始位置为 $(20000, 0, 2000)$ ，无人机初始位置为 $(17800, 0, 1800)$ ，真目标半径为 7m ，高为 10m ，导弹速度为 300m/s ，无人机速度为 120m/s ， $t = 1.5\text{s}$ 后投放烟幕干扰弹，再经过 3.6s 后烟幕干扰弹爆炸，起爆后云团半径 10m ，以 3m/s 速度下沉，在 20s 内可以进行有效遮挡。

在第一阶段 $t = 0$ 到 $t = 1.5\text{s}$ 时，按照无人机飞行模型以及导弹飞行模型求解出 $t = 1.5\text{s}$ 时两者的位置坐标。在第二阶段 $t = 1.5\text{s}$ 到 $t = 5.1\text{s}$ 时，按照导弹运动模型以及烟幕干扰弹运动模型求解出在 $t = 5.1\text{s}$ 时两者的位置坐标。在第三阶段 $t = 5.1\text{s}$ 到 $t = 25.1\text{s}$ 时，我们取微小时间间隔为 0.0001s 并在 20s 内进行遍历微小时间片段进行判断，将满足完全遮蔽条件的时间片段相加得到最终答案。我们求解出的答案为 1.39s 。

六、问题二的模型建立与求解

6.1 问题二的模型建立

6.1.1 决策变量定义以及运动轨迹建模

本问题的优化目标是通过选择无人机飞行策略和投放起爆烟幕干扰弹的时间参数,最大化对导弹 M1 的有效遮蔽时长。四个决策变量的约束如下:

1. 无人机飞行方向角 α : 无人机在水平面上的飞行方向与 x 轴正方向的夹角, 其飞行方向单位向量为 $\mathbf{d} = (\cos \alpha, \sin \alpha, 0)$ 。
2. 无人机飞行速度 v : 满足约束 $70 \leq v \leq 140$ (m/s)。
3. 烟幕干扰弹投放时间 $t_{\text{投放}}$: 从时间零点 ($t = 0$) 到无人机投放所经历的时间, 满足 $t_{\text{投放}} \geq 0$ 。
4. 烟幕干扰弹起爆时间 $t_{\text{起爆}}$: 从时间零点 ($t = 0$) 到烟幕干扰弹起爆所经历的时间, 满足 $t_{\text{起爆}} \geq t_{\text{投放}}$ 。

6.1.2 搜索方法选择

鉴于本问题中四个参数构成的决策空间规模庞大, 若直接应用启发式算法将面临较高的计算负荷。为进一步提升搜索效率并确保解的全局最优性, 本文结合无人机投弹任务的具体物理过程与运动学几何关系, 对问题进行深入分析, 通过理论推导明确最优解可能存在的子空间, 从而将参数范围约束于一个有限且物理意义明确的区域。该策略不仅大幅压缩了搜索维度, 也赋予了优化过程更强的可解释性。

在此基础上, 我们使用 Python 对经过严格约束的参数空间进行全局网格搜索。该方法能够在有限计算资源下, 系统而完备地遍历所有可能解, 最终准确定位全局最优解。

6.1.3 圆柱轨迹模型建立

我们对需要搜索的四个参数——飞行速度、飞行方向、投放时间以及起爆时间进行了一定的变换, 使得搜索更加便利。

我们将烟幕弹从无人机投放至起爆前的运动过程——无人机携弹飞行与弹体平抛运动两个阶段统一进行考虑。设该过程总时长为 T , 无人机携弹飞行时长为 t_1 , 弹体平抛运动时长为 t_2 , 则有 $T = t_1 + t_2$ 。

观察到烟幕弹在起爆前始终具有水平方向的速度, 因此若确定无人机飞行速度大小 V , 则无人机起爆前水平方向飞行距离只与 T 有关。又因为航向角 α 未确定, 因此所有可能的起爆点在水平方向上的投影是一个半径 $R = VT$ 的圆。烟幕弹起爆前的下落高度只与 t_2 的时长有关。设 $t_2 = \lambda T, 0 \leq \lambda \leq 1$, 则烟幕弹的下落高度 $h = \frac{1}{2}g(\lambda T)^2$, 也就是 $0 \leq h \leq \frac{1}{2}gT^2$ 。

因此，烟幕弹所有可能的起爆点都位于一个圆柱面上。该圆柱面的圆心位于无人机的初始坐标点 $(17800, 0, 1800)$ ，半径 $R = VT$ ，高度 $h = \frac{1}{2}gT^2$ ，顶部高度与无人机高度相同。这个圆柱面的公式为：

$$\begin{cases} (x - x_{FY1})^2 + (y - y_{FY1})^2 = (VT)^2 \\ \max(0, z_{FY1} - \frac{1}{2}gT^2) \leq z \leq z_{FY1} \end{cases} \quad (19)$$

当无人机的飞行速度大小 V 和起爆前总时间 T 确定时，这个圆柱面就精准刻画了所有的可能起爆点位置，实现了决策空间的降维，大幅减少了计算量，方便后续分析。由于导弹飞行速度固定，因此还可以计算出导弹的坐标位置。同时，如果知道了圆柱面上某一起爆点的坐标 (x, y) ，也可以很容易的反推出 t_1, t_2 的时长，之后便可计算烟幕弹投放点 (x_1, y_1) 的坐标。具体反推公式如下：

$$\begin{cases} t_2 = \sqrt{\frac{2(z_{FY1} - y)}{g}} \\ t_1 = T - t_2 \end{cases} \quad (20)$$

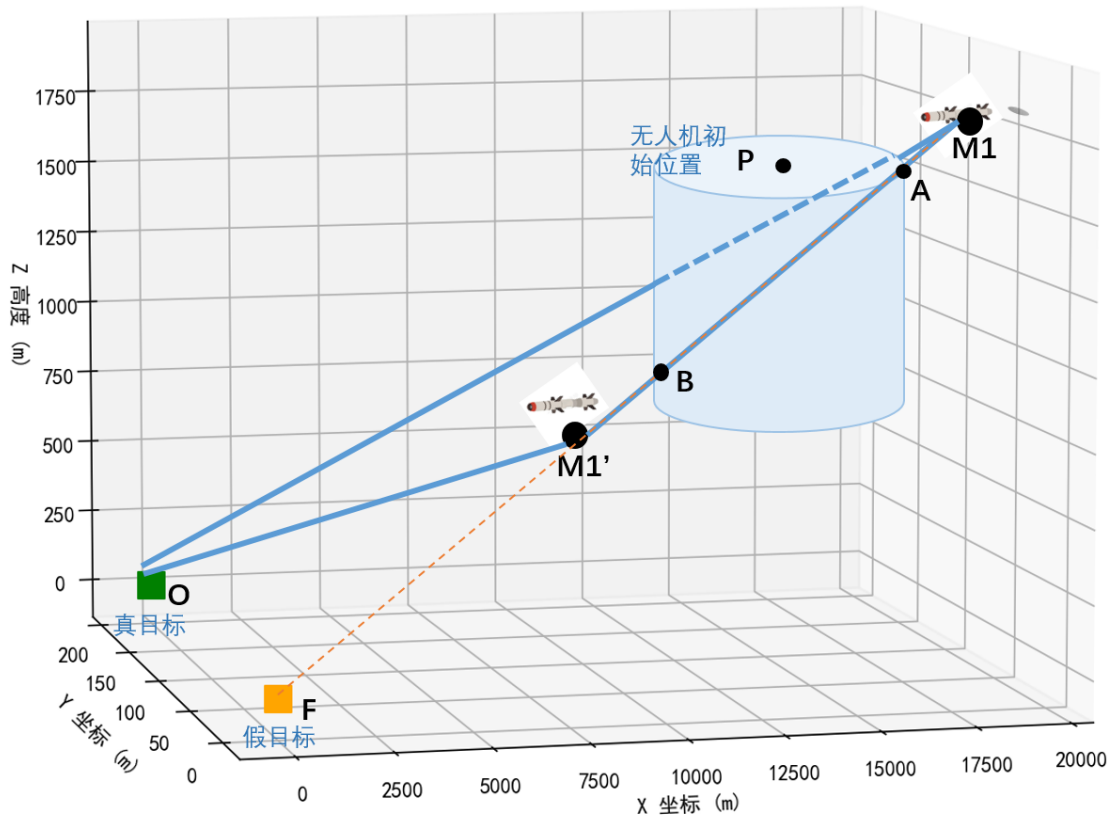


图6 圆柱轨迹模型示意图

我们还可以进一步对圆柱面上的起爆点进行筛选。容易发现，只有那些位于导弹和真目标视线连线范围内的起爆点才有更大概率干扰更长时间。对于不在视线范围内的

点，可以直接排除。因此我们可以通过几何计算来对圆柱面上的点划分范围，进而精确目标起爆点的范围。若固定无人机的速度大小为 V ，如图7所示，可以发现对于不同的 T 一共会出现三种情况：导弹未飞入圆柱面，导弹飞进圆柱面，导弹飞出圆柱面。我们可以针对这三种情况分别讨论。对于每一种情况，根据无人机的视线连线范围来选择圆柱面对应弧度的面，从而筛选有效的爆炸点，排除无效的爆炸点。

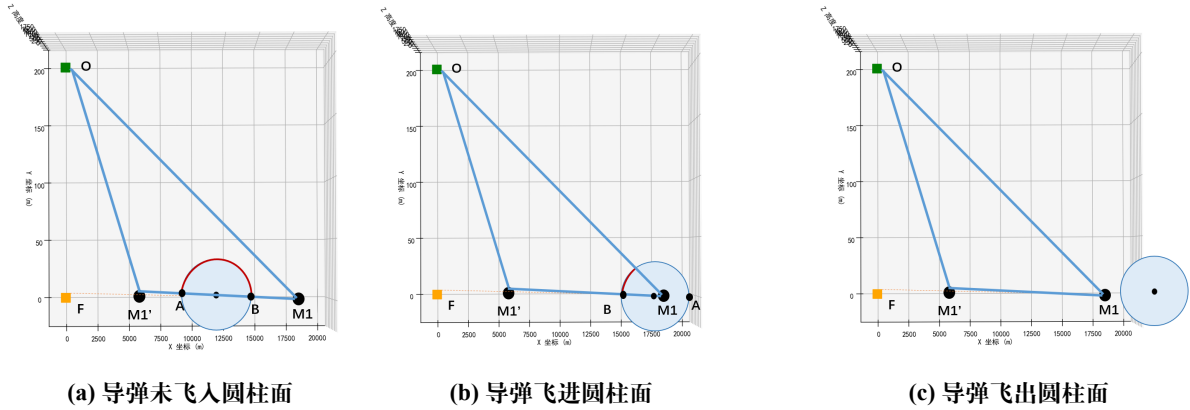


图 7 三种位置情况

俯视图分析：如图8所示，以导弹未飞入圆柱面情况为例，将三角形 $\triangle M_1OM_1'$ 及圆柱投影至 XOY 平面。圆的投影与导弹轨迹 M_1M_1' 产生 A、B 两个交点。分析可知，若导弹在 AB 段上运动，且烟幕干扰弹位于 AB 向 O 一侧的圆弧，则可实现对导弹视线的有效遮蔽；反之，若烟幕干扰弹投放于 AB 向 O 反侧的圆弧，则无法遮蔽目标。因此，无人机的航向角与航速的取值范围被约束于图中红色线框所示的圆柱侧面区间。该空间限定为后续参数遍历与最优遮蔽策略求解划定了答案的可行域，在此空间内搜索可以大大减小计算量。

对于第三种情况，导弹飞出了圆柱面，此时无论在哪里起爆都无法遮蔽导弹，因此所有点都无效。为了防止这种情况发生，我们对起爆前时间 T 的最大值 T_{max} 有约束要求：

$$T_{max} \times (300 - V) \leq d_{M_1FY_1} + R \quad (21)$$

这是一个简单的相遇问题。只要满足上式，就可以保证只会出现第一种和第二种情况。

6.1.4 圆柱面模型求解思路

当无人机速度大小 V 和起爆前时长 T 确定时，我们可以计算出当前圆柱面模型和导弹的位置，并进一步根据导弹视线范围确定圆柱面上的有效弧面，然后根据搜索算法在弧面上寻找最佳起爆点，使得有效干扰时长最大。

因此，我们可以遍历 V 和 T ，然后对每种情况计算最佳起爆点坐标，并反推烟幕弹投放点坐标和航向角。最后选出全局最优解作为答案。

6.2.1 求解结果

经过求解，当无人机的飞行速度大小为 $125m/s$ ，航向角为 5.04° ，单位方向向量为 $(0.9959, 0.0904)$ ， $t_1 = 0.753s$ ， $t_2 = 0.247s$ 时，最大的有效遮蔽时长为 $4.60s$ 。在以无人机飞行方向为 x 轴，烟幕弹下落方向为 y 轴的坐标系中，我们还绘制了无人机有效遮蔽时间与最佳起爆点和无人机起点距离的图像，如图9。从图像中可以看出，当起爆点与无人机起点相距 $125m$ 左右时，有效遮蔽时间最长，此时正好是无人机起飞后 $1s$ 的时刻，与我们得出的结果是相符的。

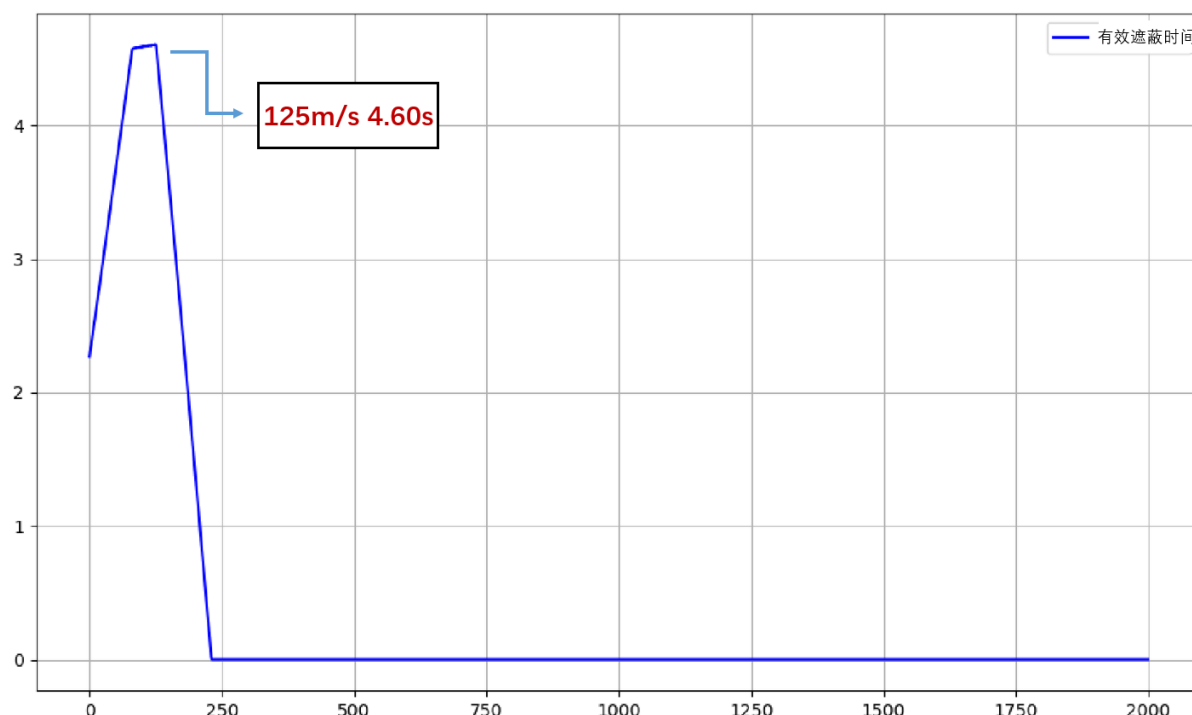


图9 遮挡效果投影俯视图

七、问题三的模型建立与求解

7.1 问题三的模型建立

7.1.1 思路剖析

本问题基于第二问，将考虑一枚烟幕弹的投放扩展成了三枚烟幕弹的投放优化，这使得待确定的参数量大大增加。使用如粒子群优化算法等启发式算法的计算量不可估量，且陷入局部最优解的风险也大大增加。因此，我们更需要深度剖析题目内在的几何特性以及物理规律，对可行域进行更加严格的约束，在降低计算量的同时能够搜索到全局最优解。因此我们解题的关键在于挖掘最优遮蔽位置的几何关系。

7.1.2 最优遮蔽模型建立

本问题要求探讨三枚干扰弹的投放策略，我们先从高层视角分析一枚干扰弹何时起爆能够遮挡住目标。首先，在这一几何计算的过程中暂时将真目标、云团和导弹视为质点，以方便求解直线方程。很容易能想到，当真目标、云团与导弹三点共线的时候有效遮蔽时长最大。如图10, 真目标 F，云团 S 与导弹运动到某一处的位置 M1' 三点共线，此时云团能够最大发挥遮蔽效果。

基于上述观察，我们希望求解出最佳的烟幕弹起爆线——在这条线上起爆的烟幕弹能够保证经过一定的下落时间后，其产生的云团存在与真目标和导弹三点共线的时刻。下面将详细介绍这条烟幕弹最佳起爆线的推导和验证过程。

容易发现当无人机速度和航向确定后，烟幕弹和云团始终与无人机处于一个平面内。如图10，我们随意选取无人机 P 的一个速度方向，将这个速度方向与 z 轴形成的平面表示出来（图中黄色平面），烟幕弹和云团就被约束在这个平面内。为了方便计算，在这个平面上以无人机速度方向为 x 轴，烟幕弹下落方向为 y 轴，建立了一个坐标系 xoy，称其为无人机坐标系。

在导弹 M1 朝向假目标 O 飞行的过程中，连接导弹与真目标形成视线连线，这条线与无人机平面的交点就是最佳云团点，此时云团正好遮蔽了导弹的视线。整个过程中所有视线连线组成了一个平面，也就是 $\triangle OTM_1$ 所在平面。视线平面与无人机平面的交线（如图中虚线所示）就是云团出现的最佳位置的集合。

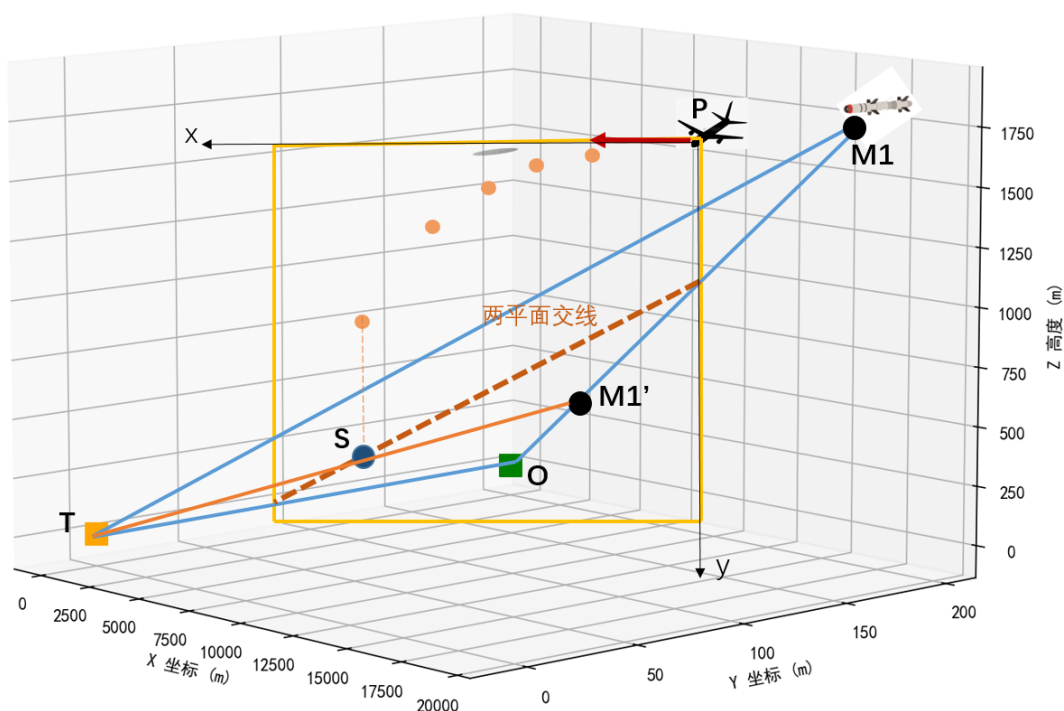


图 10 起爆场景全景图

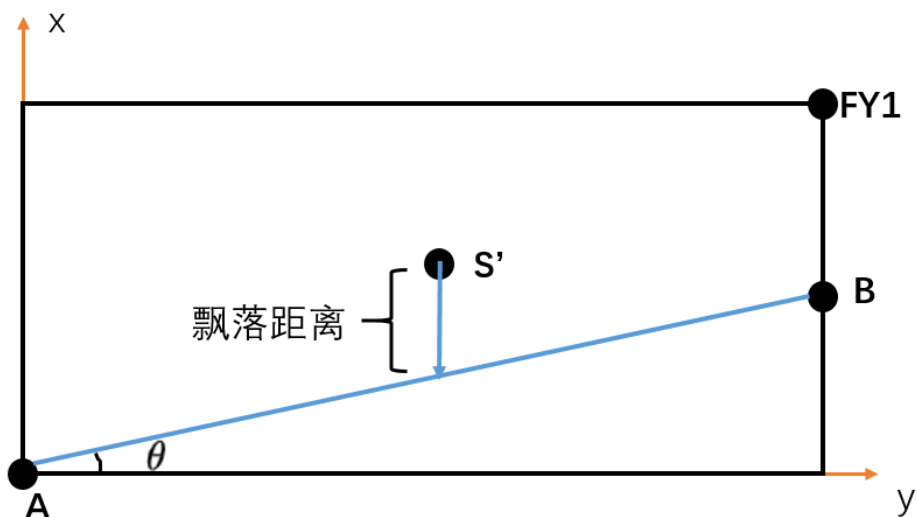


图 11 烟幕弹运动平面图

不妨称两平面的交线为云团位置线。下面将求解出云团位置线的方程：如图11，在

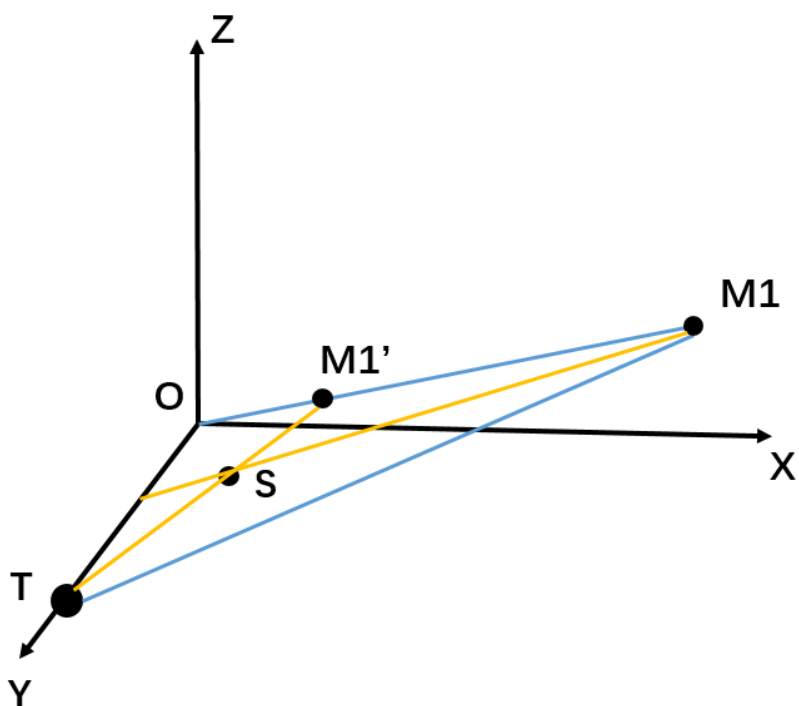


图 12 烟幕弹运动平面图

烟幕弹运动的平面图中，AB 是两个平面的交线。设 AB 与底面 (XOY 平面) 的夹角是 θ , 无人机的运动方向与 x 轴负方向的夹角为 α 。有：

$$y_B = z_{FY1} - \tan \theta \cdot x_{FY1} = 20 \quad (22)$$

解得 $B(0, 20)$ 。

$$x_A = \frac{x_{FY1}}{\cos \alpha} \quad y_A = z_{FY1} \quad (23)$$

故直线 AB 的方程为：

$$y = \frac{\cos \alpha}{x_{FY1}}(z_{FY1} - n - 20)x + 20 \quad (24)$$

S' 为烟幕弹起爆的时刻，通过运动一定的飘落距离后到达 AB 线上。由 AB 方程得飘落距离为：

$$s_{fall} = 20 + \frac{\cos \alpha}{x_{FY1}}(z_{FY1} - 20)x - y \quad (25)$$

飘落时间可以表示为：

$$t_{fall} = \frac{s_{fall}}{v_{smoke}} = \frac{20 + \frac{\cos \alpha}{x_{FY1}}(z_{FY1} - 20)x - y}{3} \quad (26)$$

接下来求解最佳起爆线的方程。如图12, 在导弹所在的平面 OTM1 上，导弹运动一定时间后的位置为 M1'，云团出现在 S 处。有：

$$\mathbf{OT} + k\mathbf{TS} = \mathbf{OM1'} \quad (27)$$

因为 OM1 处于 XOZ 平面上，所以在向量 **OM1** 上 y=0。在 y 分量上有：

$$y_{FY1} + k(x_{FY1} - \sin \theta - y_{FY1}) = 0 \quad (28)$$

解得：

$$k = \frac{y_{FY1}}{y_{FY1} - x_{FY1} - \cos \theta} \quad (29)$$

解得 M1' 点坐标为：

$$\left(x_{FY1} + k(x_{FY1} - x \cos \theta - x_{XF1}), 0, z_{FY1} + k \left(z_{FY1} - 20 - \frac{\cos \theta}{x_{FY1}}(z_{FY1} - 20x - z_{FY1}) \right) \right) \quad (30)$$

导弹需要运动 $|M1' - M1| = \frac{x}{v}v_{M1}$ ，其中 v 为无人机的速度。其中：

$$|M1' - M1| = \frac{x_{M1} - x_{M1'}}{\cos \theta} \quad (31)$$

导弹运动的时间需要等于云团从起爆最佳线飘落的时间：

$$t_{fall} = \frac{M1M1'}{v_{M1}} - \frac{x}{v} \quad (32)$$

由公式32可以求解出起爆最佳线的方程：

$$y = \frac{3x}{v} + 20 + \frac{\cos \alpha}{x_{FY1}}(z_{FY1} - 20)x + 3 \cdot \frac{x_F + \frac{y_F}{y_F - x \sin \alpha} \cdot (x_{FY1} - x \cos \alpha - x_F) - x_M}{\cos \alpha \cdot v_M} \quad (33)$$

7.1.3 实验结果分析

经过上述推导运算，我们已经得出了在无人机坐标系中，最佳起爆线的函数表达式 (33)。这条线的含义是，如果烟幕弹在位于这条线上时引爆，那么一定存在一个时刻，使得烟幕弹在匀速下落过程中与真目标、导弹三点一线，此刻正好可以完美遮蔽导弹视

线。为了验证我们的推导是否正确，我们选取了无人机速度为 120m/s 且飞行速度与 x 轴负方向夹角为 0 时的情况进行模拟实验，并可视化有效遮蔽时间结果，最后用实际情况来证实最佳起爆线是完全正确的。

我们首先可视化了函数。如图13，在无人机坐标系中，蓝色线 L1 为最佳起爆线，绿色线 L3 为云团位置线，红色线 L2 为在 $t = 0$ 时刻直接抛出导弹后的抛物轨迹线。红

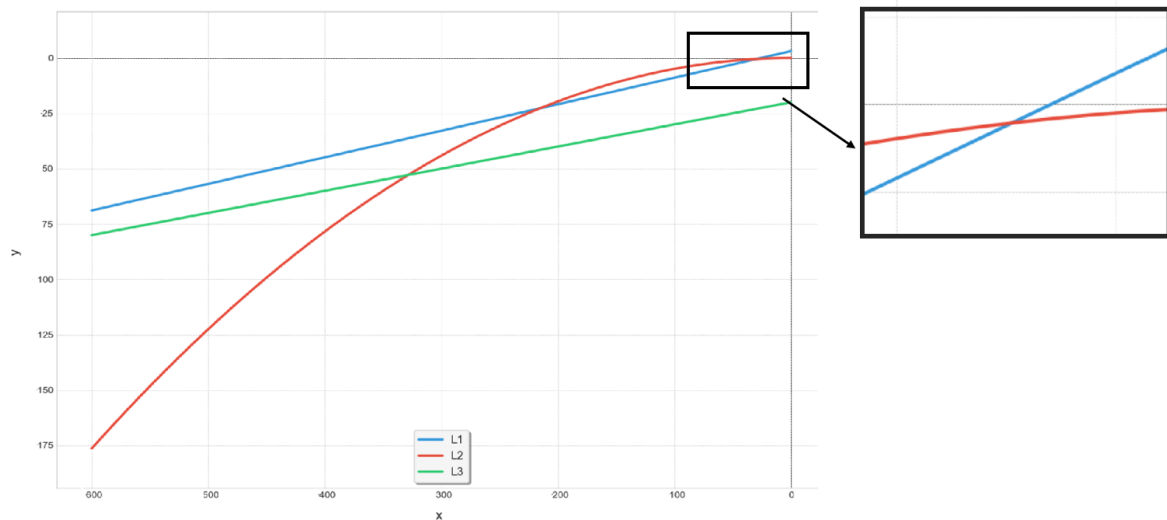


图 13 云团位置线，最佳起爆线以及抛物线

色线 L2 的作用是为了限制起爆点的范围。由于烟幕弹的下落过程是平抛运动，因此在 $t = 0$ 时刻直接抛出烟幕弹，其运动轨迹就是烟幕弹所能到达的最低点。因此 L2 线及以上部分就是所有烟幕弹所能到达的坐标点。同样根据公式 (20)，我们可以用起爆点坐标来反推烟幕弹的抛出时间与抛出点。

图13中蓝色线是最佳起爆线，绿色线是云团位置线。我们尽可能让烟幕弹在到达最佳起爆线的时候起爆，然后云团会匀速下落，等云团落至云团位置线时，此刻云团、导弹、真目标正好三点一线，这样有效遮蔽时间更长。

根据上述推导，我们可以进行模拟实验。遍历 L2 上方的所有可能起爆点，并算出每个起爆点位置的有效遮蔽时间，根据时间长短绘制热力图。如果我们的推导是正确的，那么在热力图中只有最佳起爆线附近的点有效遮蔽时间更长，其余点应该没有遮蔽时间。

实验结果如图14所示，图中出现了有效遮蔽时间不为 0 的线段，这就是最佳起爆线！这一实验结果有效且充分地证明了我们的猜想是完全正确的——确实只有云团落在这一条直线上时才能对真目标进行遮蔽。

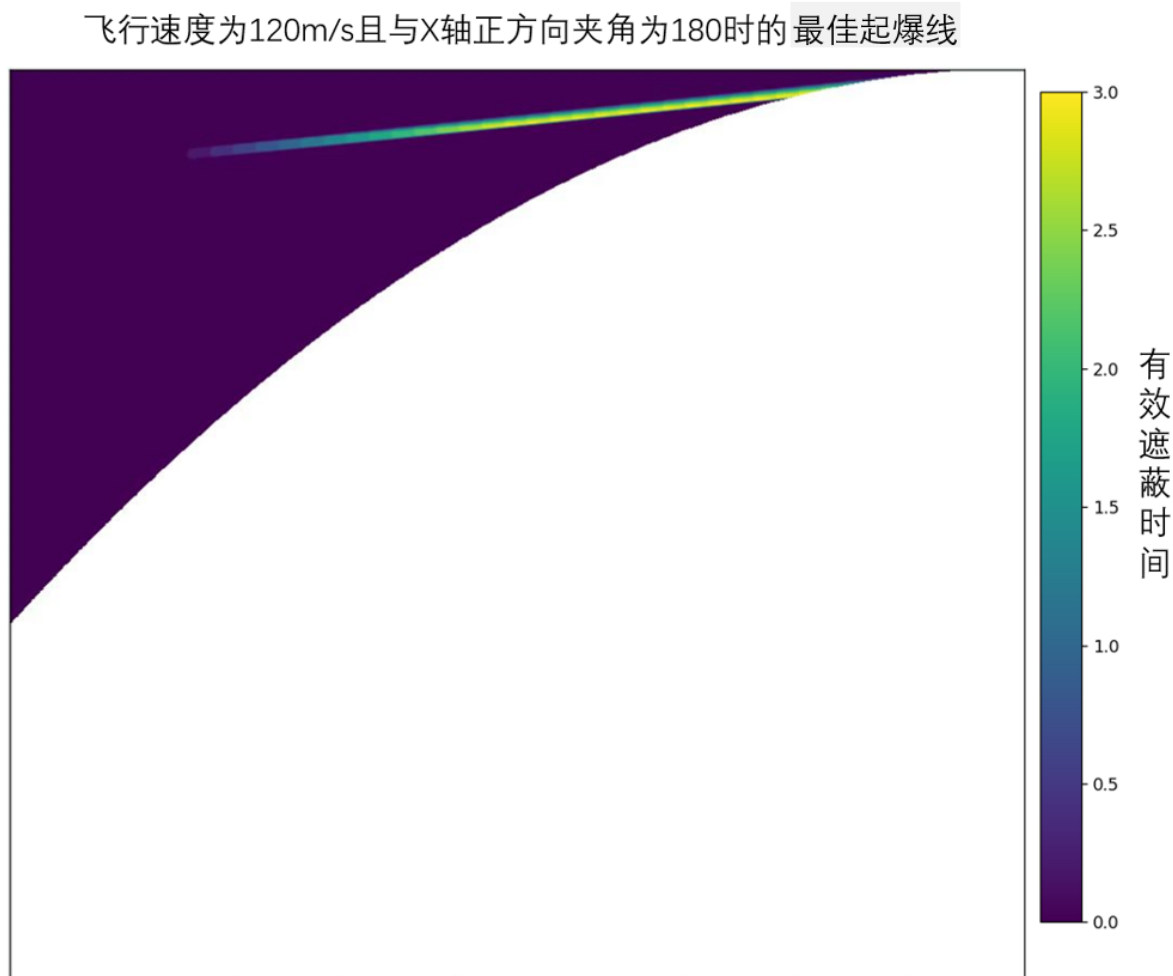


图 14 烟幕弹运动平面图

7.2 问题三的模型求解

7.2.1 模型求解

基于以上的几何变换与分析，我们求得了在无人机速度 V 和航向角 α 确定时无人机的最佳起爆线。现在需要对最佳起爆线上的各个点进行分析，探究它们的有效遮蔽时间的变化规律。如图15所示，我们选取了其中当 $v=120\text{m/s}$, $\alpha = 0.1^\circ$ 时的有效遮蔽时长结果进行展示。

图中的结果与我们的理论推导非常吻合：

1. 初始一小段时间内，当最佳起爆线位于抛物线上方时，有效遮蔽时长较大。
 2. 当最佳起爆线暂时位于抛物线下方时，由于无法使得云团落在云团位置线上，有效遮蔽时长下降，且呈现先下降再上升的趋势。
 3. 当最佳起爆线再次处于抛物线上方时，有效遮蔽时长再次上升达到一个最高点。
 4. 如图13，当最佳起爆线 $L1$ 位于云团位置线 $L3$ 下方后，有效遮蔽时长下降直至为 0。
- 因此，考虑投放三枚烟幕弹时，我们决定将第一枚烟幕弹在初始一小段时间内——当最

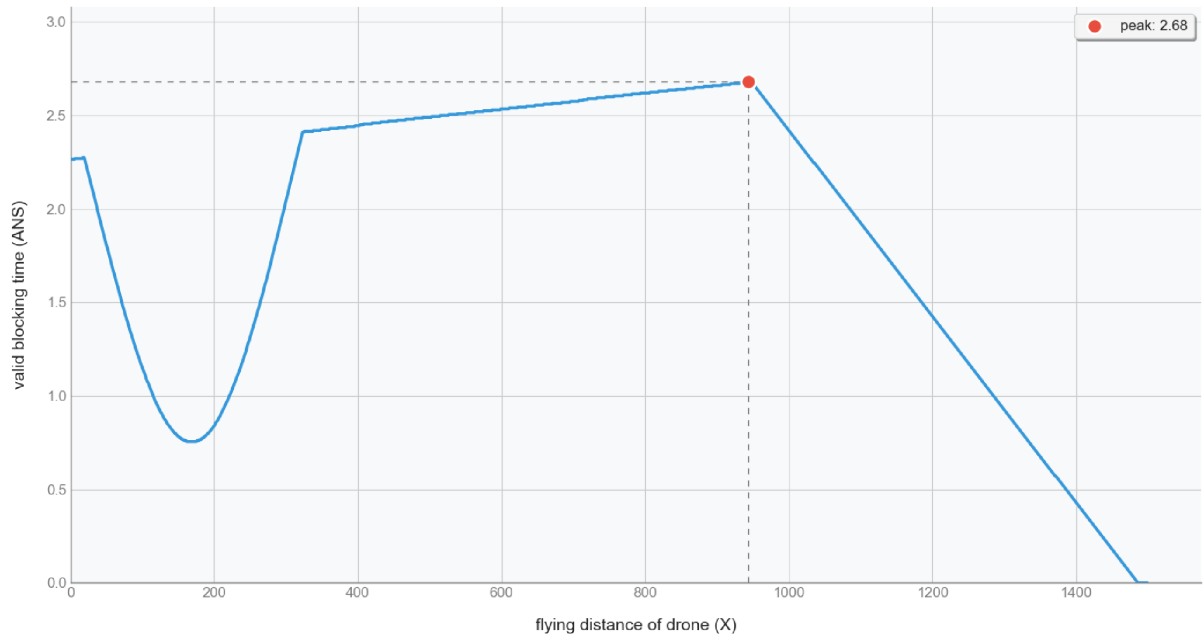


图 15 $v=120\text{m/s}, \alpha = 0.1^\circ$ 时有效遮蔽时长结果

佳起爆线位于抛物线上方时起爆；对于第二和第三枚烟幕弹，我们将于最佳起爆线再次位于抛物线上方且有效时长不为 0 的某两处投放。

约束条件：

三枚烟幕弹的投放需要满足以下三点的约束以及优化条件：（对于每一枚烟幕干扰弹 i , 设其此时的位置为 (x_i, y_i) , 投放时间为 t_1 , 起爆时间为 t_2 , 该枚烟幕干扰弹的有效遮蔽时长为 $[l_i, r_i]$ ）

1. 与上一枚烟幕弹的投放时间间隔大于 1 秒：

$$\frac{x_i}{v} - \sqrt{\frac{2y_i}{g}} \geq \frac{x_{i-1}}{v} - \sqrt{\frac{2y_{i-1}}{g}} + 1 \quad (34)$$

2. 尽可能在上一枚烟幕弹的有效遮蔽时间结束之后再进行有效遮蔽，极力避免有效遮蔽时长的重叠：

$$r_{i-1} - \frac{\cos \alpha}{x_{FY1}}(z_{FY1} - 20)x_i + 20 - y_i)/3 < \frac{x_i}{v} \quad (35)$$

3. 此条件是第二个条件的细化判断，条件二是判断当云团落在云团位置线上时的有效遮蔽时长，条件三是判断云团是否出现未落在云团位置线上时就出现遮挡的情况：在这一步判断中我们将 t_{end} 代入问题一提出的完全遮蔽模型，检查此时云团是否已经开始有效遮蔽目标。这一条约束使得我们的计算误差更小。

r_{i-1} 时刻云团的坐标可能有三种情况：

$$\left\{ \begin{array}{l} (x_{FY1} - vr_{i-1} \cos \alpha, y_{FY1} + v \cdot r_{i-1} \sin \alpha, z_{FY1}) \\ \qquad \qquad \qquad r_{i-1} < t_1 \\ (x_{FY1} - vr_{i-1} \cos \alpha, y_{FY1} + v \cdot r_{i-1} \sin \alpha, z_{FY1} - \frac{1}{2}g(r_{i-1} - t_1)^2) \\ \qquad \qquad \qquad t_1 < r_{i-1} < t_1 + t_2 \\ (x_{FY1} - v(t_1 + t_2) \cos \alpha, y_{FY1} + v(t_1 + t_2) \sin \alpha, z_{FY1} - \frac{1}{2}gt_2^2 - 3(r_{i-1} - (t_1 + t_2))) \\ \qquad \qquad \qquad r_{i-1} > t_1 + t_2 \end{array} \right. \quad (36)$$

M1' 坐标为 $(x_{M1} - v_{M1} \cos \alpha, 0, z_{M1} - v_{M1} \sin \alpha)$,

检查此时云团是否能够对 M1' 完全遮蔽。

需要最大化的目标为：

$$\max T_{\text{eff}} = \int_{t_0}^{t_f} (\eta = 1) dt \quad (37)$$

我们采用二分查找法寻找出三枚烟幕弹的投放位置。即当烟幕弹所处位置满足上述三个条件时选择该位置进行投放。

7.2.2 求解结果

最后经过对参数的全局搜索并寻优的计算后，我们得出以下结果：

表 1 无人机烟幕干扰弹投放参数与效果

运动方向	烟幕干扰弹	投放点坐标 (m)			起爆点坐标 (m)			有效干扰	有效遮蔽
(m/s)	编号	x	y	z	x	y	z	时长 (s)	区间
140	1	17800	0	1800	17800	0	1800	2.56	[4.85, 7.40]
140	2	17500	1.05	1800	16845.7	3.33	1692.95	3.18	[7.43, 10.60]
140	3	17133.94	2.33	1800	16315.06	5.18	1632.36	1.77	[10.61, 12.37]

八、问题四的模型建立与求解

8.1 问题四的模型建立

如图16所示，在本问题中我们需要综合考虑三架无人机分别投放一枚烟幕弹的最优策略，这涉及到时间以及空间上双重的协同优化。对于三架位置不同的无人机，理想的

最优策略是三者各司其职，实现高效的“分段遮蔽干扰战术”。由图16可以发现三架无人机可以基于各自的初始位置优势，在时空上明确分工，形成覆盖导弹飞行全程的立体拦截网。

单独考虑每一枚烟幕弹时思路可以延续问题二建立的“圆柱轨迹模型”，当考虑多枚烟幕弹时就需要加入时序上的协调，因为每一枚烟幕弹的最优解合并不一定是三枚烟幕弹的全局最优解。因此加上最小化重叠有效遮蔽时长的约束：

设 FY_i 的有效遮蔽时间段为 $[t_{i,start}, t_{i,end}]$ ，则不存在 i, j ，满足

$$t_{i,start} \leq t_{j,start} \leq \min(t_{i,end}, t_{j,end})$$

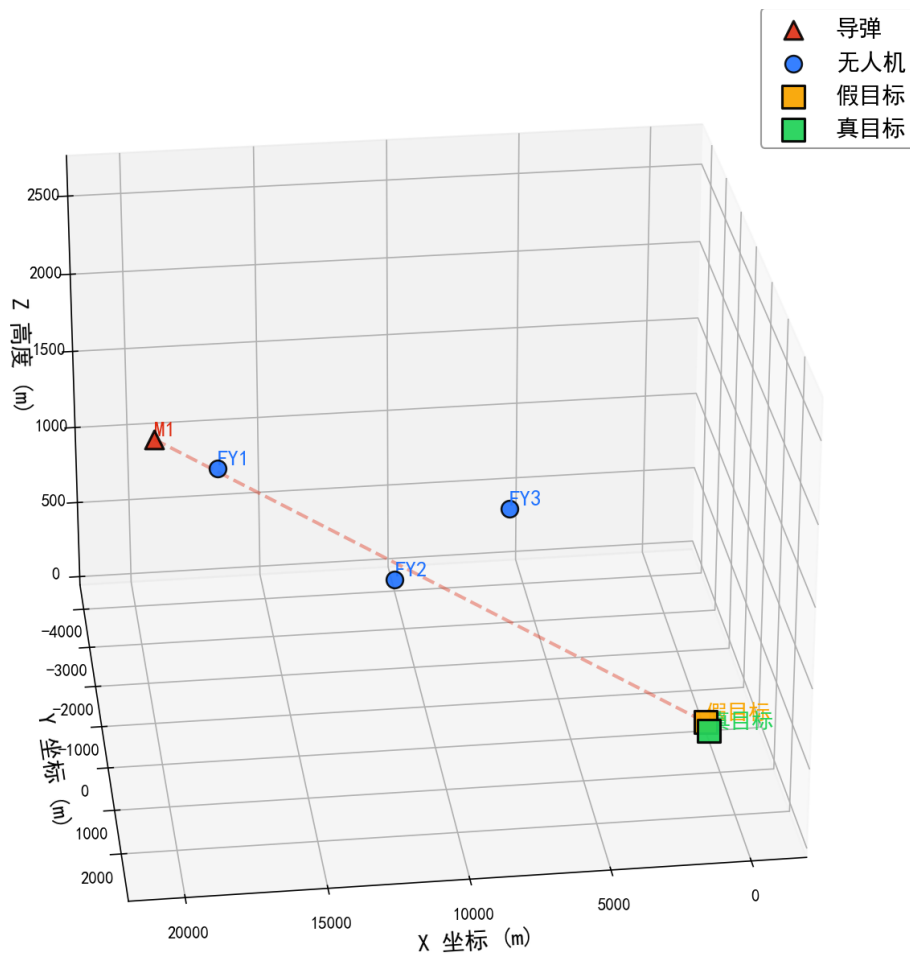


图 16 问题四全景模型图

8.2 问题四的模型求解

经过在多约束条件下的搜索求解，我们得出如图17所示的结果。三架无人机投放烟幕弹的有效遮蔽时长分别为：[1.07, 5.66], [10.76, 14.77], [24.84, 28.04]。实验结果验证得

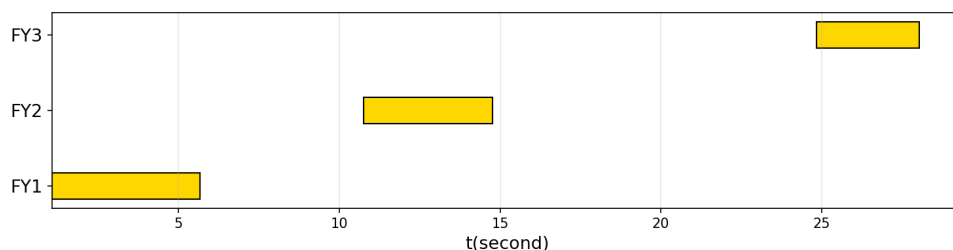


图 17 问题四结果示意图

出，三架无人机最优情况下分别投放一枚烟幕弹对 M1 的有效遮蔽时长并不重叠，且对于每一枚烟幕弹都是最优解。因此这一投放策略可以得到三枚烟幕弹有效遮蔽时长的全局最优解。

FY1 为“早期拦截者”，由于它与导弹的初始距离较小且与导弹-真目标连线较近，因此负责对导弹飞行初期进行干扰。FY2 负责中期拦截，精准遮蔽导弹飞行的航线中段。FY3 进行末期拦截，从导弹航线远端侧前方触发，伺机而动，在任务末端投放烟幕弹，对临近目标区域实施决定性干扰。综上，我们得到的答案如下：

表 2 无人机烟幕干扰弹投放参数与效果

无人机编号	运动方向 (°)	速度 (m/s)	投放点坐标 (m)			起爆点坐标 (m)			有效干扰时长 (s)
			x	y	z	x	y	z	
FY1	5.04	125	17893.74	85.1	1800	17924.49	11.3	1799.7	4.62
FY2	285.01	140	12159.33	805.82	1400	12362.6	47.74	494.42	4.01
FY3	82.66	130	6345.45	-319.08	700	6398.74	1246.01	649.6	3.15

九、问题五的模型建立与求解

9.1 问题五的模型建立

在这个问题中，我们的目标为最大化三枚导弹的平均有效遮蔽时长。可以先将本问题视为问题三的扩展，基于“最佳遮蔽模型”我们可以猜想，每架无人机对每一枚导弹都存在最佳起爆线以及云团最佳位置线。图18摘取了无人机 FY4 在 140m/s 的飞行速度下和 100° 的飞行方向对应 3 枚导弹云团位置线中的示意图。如图可以看到图中有 3 条较为明显的云团位置线。

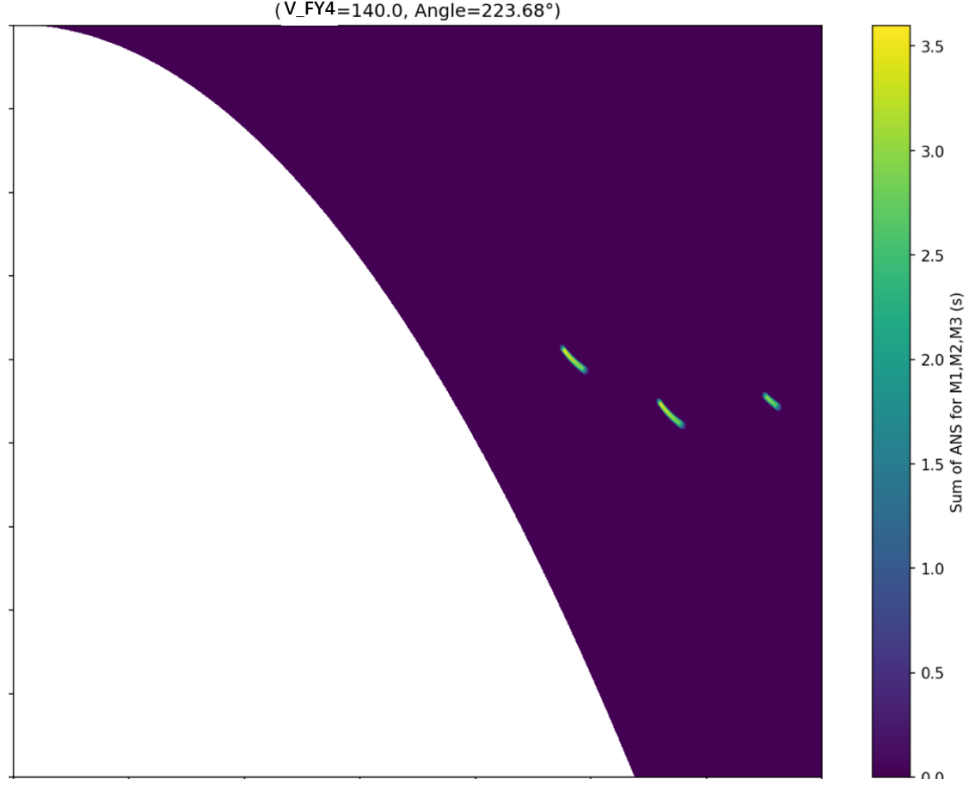


图 18 三条最佳起爆线示意图

9.2 问题五的模型求解

接下来，基于猜想和实验验证，对于每架无人机 FYi 和每枚导弹 Mi, 我们求解出最佳起爆线以及云团最佳位置线的通解：导弹的视线平面可以表示为：

$$S_1 : -z_{Mi}x + x_{Mi}z = 0 \quad (38)$$

无人机所在的运动平面可以表示为：

$$S_2 : (x_{FYi} - x)\sin\alpha + (y_{FYi} - y)\cos\alpha = 0 \quad (39)$$

联立 S_1 和 S_2 平面得到两面交线：

$$\begin{cases} x = t \\ y = \frac{(x_{FYi} - t)\sin\alpha}{\cos\alpha} + y_{FYi} \\ z = \frac{z_{Mi}}{x_{Mi}}t \end{cases} \quad (40)$$

其中，

$$t = x_{FYi} - x\cos\alpha \quad (41)$$

由三点一线的条件联立得 M'_i 的坐标可以表示为：

$$(kt, 200 + k(\frac{(x_{FYi} - t)\sin\alpha}{\cos\alpha} + y_{FYi} - 200), \frac{z_{Mi}}{x_{Mi}}t) \quad (42)$$

因为有共线关系

$$OM \parallel OM'$$

，进一步联立可以解得 k 的表示为：

$$k = \frac{200}{\frac{y_{Mi}}{x_{Mi}}t - \frac{(x_{FYi}-t)\sin\alpha}{\cos\alpha} + y_{FYi} - 200} \quad (43)$$

因此最佳起爆线的通解方程为：

$$\frac{3|M'M|}{V_{Mi}} - \frac{3x}{V} = z_{FYi} - (x_{FYi} - x\cos\alpha)\frac{z_{Mi}}{x_{Mi}} - y \quad (44)$$

解决问题五同时也需要多架无人机之间的时序协同投放安排，因此我们将问题四中最小化重叠有效遮蔽时间的约束也加入问题五的求解中：设 FY_i 的有效遮蔽时间段为 $[t_{i,start}, t_{i,end}]$ ，则不存在 i, j ，满足

$$t_{i,start} \leq t_{j,start} \leq \min(t_{i,end}, t_{j,end})$$

经过实验可视化结果后，我们发现 $FY2, FY3, FY4, FY5$ 的最佳起爆线非常短，如图19所示。因此对于这四架无人机，不可能出现一条最佳起爆线上起爆两枚烟幕干扰弹，即不可能让同一架无人机的两枚烟幕干扰弹同时对一枚导弹进行干扰。因此我们的策略是对于每架飞机的三枚烟幕干扰弹，遍历搜索它们在三条最佳起爆线上产生最长有效遮蔽时长的位置，将它们分别安排在三条起爆线上。

采用搜索策略如下：分别对四架无人机进行一次搜索，每次搜索遍历可能的横坐标，求解三条起爆线上对应的纵坐标，从而得到三个最佳起爆点，对这三个点进行模拟得到三个有效干扰时间。最终求解出一个最大的有效干扰时间之和。并检验得到满足投放间隔需相差 1s 的要求。

对于 $FY1$ 我们进行了额外的分析，发现也无法让两枚烟幕干扰弹遮蔽一枚导弹。因此我们延用了问题三的结果。

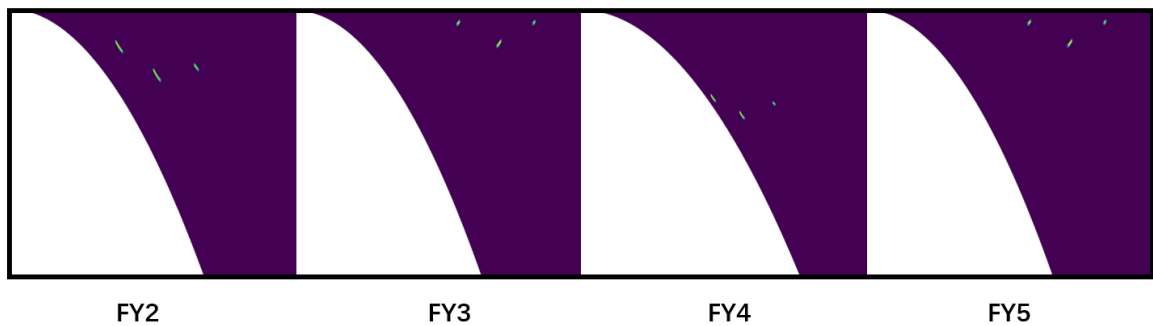


图 19 $FY2, FY3, FY4, FY5$ 最佳起爆线示意图

无人机编号	FMEA	速度 (m/s)	弹编号	投放点坐标 (m)			起爆点坐标 (m)			干扰时长
				x	y	z	x	y	z	
FY1	179.8	140	1	17800	0	1800	17800	0	1800	2.56
FY1	179.8	140	2	17500	1.05	1800	16845.7	3.33	1692.95	3.18
FY1	179.8	140	3	17133.94	2.33	1800	16315.06	5.18	1632.36	1.77
FY2	260	120	1	11908.98	883.82	1400	11826.56	416.37	1323.33	3.84
FY2	260	120	2	11886.83	758.18	1400	11756.89	21.27	1209.47	3.66
FY2	260	120	3	11806.02	299.87	1400	11687.43	-372.65	1241.31	3.62
FY3	100	130	1	5595.55	-706.28	700	5486.21	-86.15	585.03	2.84
FY3	100	130	2	5572.98	-578.25	700	5453.01	102.15	561.6	2.86
FY3	100	130	3	5515.27	-250.98	700	5426.96	249.86	625.01	2.65
FY4	280	140	1	11035.61	1798.03	1800	11286.52	375.07	1278.06	3.54
FY4	280	140	2	11069.11	1608.04	1800	11347.3	30.39	1158.41	3.67
FY4	280	140	3	11149.82	1150.35	1800	11416.76	-363.54	1209.22	3.13
FY5	120	140	1	12228.24	-663.27	1300	12063.65	-378.19	1272.91	2.31
FY5	120	140	2	12158.24	-542.02	1300	11825	35.16	1188.95	3.79
FY5	120	140	3	11719.59	217.74	1300	11625	381.57	1291.05	3.56

9.2.1 求解结果

最后我们得到的结果上表所示。

十、模型的分析与检验

由于问题三、四、五都是基于问题二变化而得，因此我们对问题二的圆柱轨迹模型进行检验，以证明模型的合理性。

10.1 灵敏度分析

我们在问题二最优参数组合下对飞行速度、飞行方向角以及投放起爆总时间进行分别的取值探索。实验结果如图20、21和22所示。

根据灵敏度分析结果可知，无人机飞行速度、投放起爆总时间及飞行方向角的变化均对遮蔽效果具有一定影响。如图17所示，当飞行速度在合理区间内增加时，有效遮蔽时间大体呈现先升后降的趋势，表明存在一个使遮蔽效能最大化的最优速度阈值，速

度过低或过高均会导致有效遮蔽时间缩短。从图 18 的时间灵敏度分析可见，投放起爆时间窗口的设定对决策变量的选择具有一定约束作用，过早或过晚的干预均会大幅降低遮蔽有效性，进一步验证了模型中对时间参数进行协同优化的必要性。图 19 的分析结果表明，飞行方向角的变化对遮蔽效果的波动较为敏感，其最优解分布在一个狭窄的连续区间内，说明该参数是影响系统性能的关键因素，需要进行精确控制。角度的敏感性也证明了“圆柱轨迹模型”中通过投影解出角度搜索范围的重要性，进一步说明了我们的模型对比启发式算法直接搜索具有显著的优势。

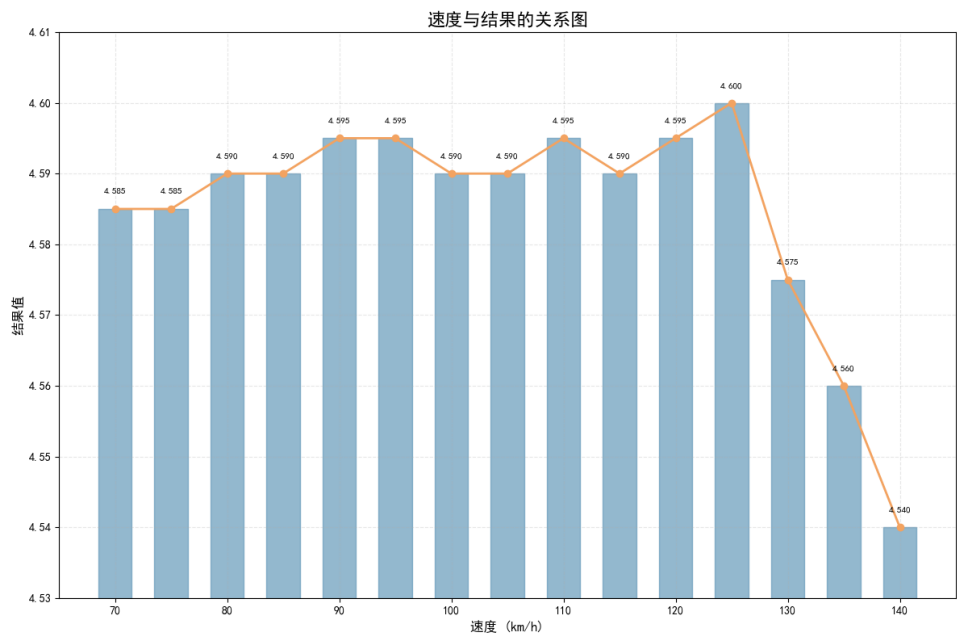


图 20 速度灵敏度分析

10.2 误差分析

基于问题一，我们对在真目标内进行蒙特卡洛取点模拟视线的处理进行了误差分析。如表3所示，随着采样点数量 N 的增加，有效遮蔽时间的估计值逐渐趋于稳定：当 $N = 1000$ 和 $N = 2000$ 时，结果为 1.3929 s；当 $N \geq 3000$ 时，结果收敛至 1.3919 s，并在 N 继续增大至 20000 的过程中保持不变。说明蒙特卡洛模拟在此问题上具有良好的数值稳定性。在 $N < 3000$ 时，结果存在的微小差异（0.001 s）源于蒙特卡洛方法固有的随机抽样误差。而当 N 超过 3000 后，该误差被有效控制在可接受的范围（小于 0.001 s）内，证明了我们选取 $N \geq 10000$ 作为后续计算的基准是充分且准确的。

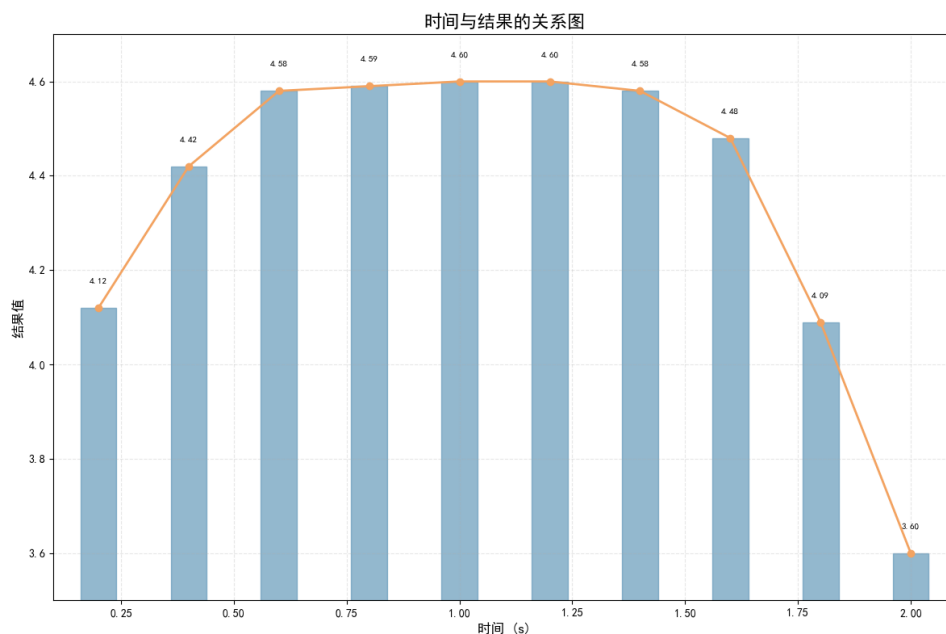


图 21 时间灵敏度分析

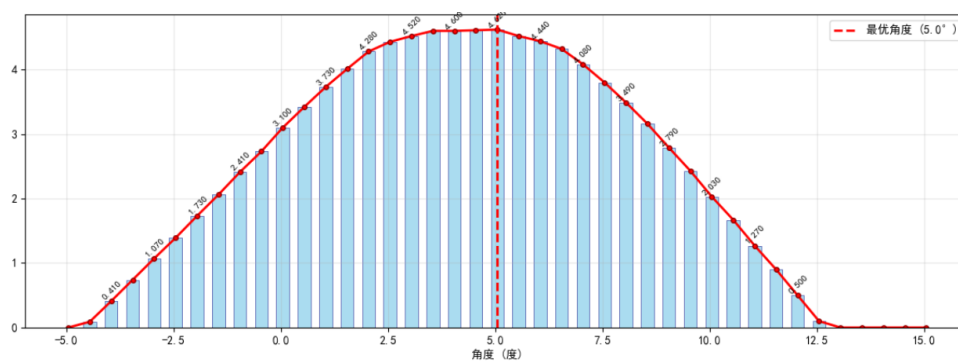


图 22 飞行方向角灵敏度分析

十一、模型的评价

11.1 模型的优点

- 充分挖掘原问题中的物理规律和几何特性，建立圆柱轨迹模型、最优遮蔽模型等以缩小答案可行域，简化题目复杂度，方法极具创新性。
- 避免直接使用启发式算法进行暴力搜索，大大降低了计算成本，提升计算效率

表 3 取点数量与有效遮蔽时间关系

取点数量 N	1000	2000	3000	5000	10000	20000
有效遮蔽时间 (s)	1.3929	1.3929	1.3919	1.3919	1.3919	1.3919

的同时有效避免陷入局部最优解。

- 经过严谨的理论推导与实验验证，双重保证答案正确性及模型健壮性，并进行了灵敏度分析。

- 模型没有对真实情况进行过多假设减省，如使用蒙特卡洛方法模拟导弹发现真目标的全部视线，而不是将真目标视为质点，因此精确地解决问题。

- 模型可以扩展应用到更多无人机和更多烟幕干扰弹的场景下，具有良好的泛化性。

11.2 模型的缺点

- 在实际条件下空间并非直角坐标系，而是球面坐标，在本文的建模中将环境简化为三维直角坐标系。

- 物体的运动均假设在无风条件下，导弹、无人机以及云团的匀速下降条件苛刻，且未考虑到烟幕云团的扩散问题。

参考文献

- [1] 司守奎, 孙玺菁. 数学建模算法与应用[M]. 北京: 国防工业出版社, 2011.
- [2] 卓金武. MATLAB 在数学建模中的应用[M]. 北京: 北京航空航天大学出版社, 2011.
- [3] 李庆民, 于志良, 王红军. 基于多阶段多指标的编队干扰方案协同决策研究[J]. 系统工程与电子技术, 2009, 31(06):1404-1408.

附录 A 文件列表

文件名	功能描述
result1.xlsx	问题三结果
result2.xlsx	问题四结果
result3.xlsx	问题五结果
problem1.py	问题一程序代码
problem2new.py	问题二程序代码
problem3.py	问题三程序代码
problem4.py	问题四程序代码
T5search.py	问题五程序代码
AI 工具使用详情.pdf	AI 工具使用详情

附录 B 代码

q1.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D
4 import math
5
6 plt.rcParams['font.sans-serif'] = ['SimHei']
7 plt.rcParams['axes.unicode_minus'] = False
8
9 def point_segment_distance(p, a, b):
10     """
11     计算点p到线段ab的距离。
12     p, a, b 均为numpy array形式的3D坐标。
13     """
14     ab = b - a
15     ap = p - a
16
17     dot_product = np.dot(ab, ab)
18     if dot_product < 1e-9:
19         return np.linalg.norm(p - a)
```



```

20
21     t = np.dot(ap, ab) / dot_product
22
23     if t < 0.0:
24         return np.linalg.norm(p - a)
25     elif t > 1.0:
26         return np.linalg.norm(p - b)
27     else:
28         projection = a + t * ab
29         return np.linalg.norm(p - projection)
30
31
32 def generate_cylinder_points(center, radius, height,
33     num_points=100):
34     """
35     在圆柱体表面进行均匀采样。
36
37     与原始方法不同，此函数根据侧面和底面的实际表面积，
38     按比例分配采样点的数量，从而实现真正的均匀分布。
39
40     参数：
41     center (np.array): 圆柱体下底面的圆心坐标 [x, y, z]。
42     radius (float): 圆柱体的半径。
43     height (float): 圆柱体的高度。
44     num_points (int): 要生成的总点数。
45
46     返回：
47     np.array: 一个 (num_points, 3) 的Numpy数组，包含点的坐标。
48     """
49     # 1. 计算各个部分的表面积
50     area_side = 2 * np.pi * radius * height
51     area_cap = np.pi * radius ** 2
52     total_area = area_side + 2 * area_cap
53
54     # 2. 根据面积比例，计算分配到每个部分的点数

```

```

54     # 使用round()确保点数为整数，并将剩余点数加到最大的区域
    (侧面)
55     num_side = int(round(num_points * (area_side / total_area)
66     ))
56     num_cap_each = int(round(num_points * (area_cap /
total_area)))
57
58     # 由于取整可能会有误差，重新分配以确保总点数不变
59     num_caps_total = num_points - num_side
60     num_top_cap = num_caps_total // 2
61     num_bot_cap = num_caps_total - num_top_cap
62
63     points = []
64
65     # 3. 在圆柱体侧面生成点
66     for _ in range(num_side):
67         # 在 [0, 2*pi] 范围内均匀选择角度
68         angle = np.random.uniform(0, 2 * np.pi)
69         # 在 [0, height] 范围内均匀选择高度
70         z = center[2] + np.random.uniform(0, height)
71
72         x = center[0] + radius * np.cos(angle)
73         y = center[1] + radius * np.sin(angle)
74         points.append([x, y, z])
75
76     # 4. 在圆柱体上底面生成点
77     z_top = center[2] + height
78     for _ in range(num_top_cap):
79         # 在 [0, 2*pi] 范围内均匀选择角度
80         angle = np.random.uniform(0, 2 * np.pi)
81         # 为了保证点在圆内均匀分布，半径r的取值需要开根号
82         # 这是因为圆的面积与半径的平方成正比
83         r = radius * np.sqrt(np.random.uniform(0, 1))
84
85         x = center[0] + r * np.cos(angle)

```

```

86         y = center[1] + r * np.sin(angle)
87         points.append([x, y, z_top])
88
89     # 5. 在圆柱体下底面生成点
90     z_bot = center[2]
91     for _ in range(num_bot_cap):
92         angle = np.random.uniform(0, 2 * np.pi)
93         r = radius * np.sqrt(np.random.uniform(0, 1))
94
95         x = center[0] + r * np.cos(angle)
96         y = center[1] + r * np.sin(angle)
97         points.append([x, y, z_bot])
98
99     return np.array(points)
100
101
102 def visualize_cylinder(points):
103     """
104     将生成的圆柱体点进行三维可视化
105     """
106
107     plt.style.use('seaborn-v0_8-whitegrid')
108
109     fig = plt.figure(figsize=(10, 8))
110     ax = fig.add_subplot(111, projection='3d')
111
112
113     scatter = ax.scatter(points[:, 0], points[:, 1], points[:,
114     2],
115                           c=points[:, 2],
116                           cmap='viridis',
117                           marker='.',
118                           s=5,
119                           alpha=0.6)

```

```

120     center_x = (points[:, 0].max() + points[:, 0].min()) / 2
121     center_y = (points[:, 1].max() + points[:, 1].min()) / 2
122     z_min, z_max = points[:, 2].min(), points[:, 2].max()
123     radius = np.sqrt((points[:, 0] - center_x) ** 2 + (points
124    [:, 1] - center_y) ** 2).max()
125
126     theta = np.linspace(0, 2 * np.pi, 100)
127     x_circle = center_x + radius * np.cos(theta)
128     y_circle = center_y + radius * np.sin(theta)
129     ax.plot(x_circle, y_circle, z_min, color='gray', linestyle
130     = '-', linewidth=1.5, label='bottom')
131     ax.plot(x_circle, y_circle, z_max, color='gray', linestyle
132     = '-', linewidth=1.5)
133
134     ax.set_xlabel('X (m)', fontsize=12)
135     ax.set_ylabel('Y (m)', fontsize=12)
136     ax.set_zlabel('Z (m)', fontsize=12)
137     ax.set_title('cylinder frame', fontsize=16, pad=20)
138     ax.legend(loc='upper left')
139
140     cbar = fig.colorbar(scatter, ax=ax, shrink=0.6, pad=0.1)
141     cbar.set_label('Z (m)', fontsize=10)
142
143     ax.xaxis.set_pane_color((1.0, 1.0, 1.0, 0.0))
144     ax.yaxis.set_pane_color((1.0, 1.0, 1.0, 0.0))
145     ax.zaxis.set_pane_color((0.95, 0.95, 0.95, 1.0))
146
147     ax.view_init(elev=25., azim=-135)
148
149     max_range = np.array([points[:, 0].max() - points[:, 0].
150                           min(),
                           points[:, 1].max() - points[:, 1].

```

```

min(),
151         points[:, 2].max() - points[:, 2].
min()]).max() / 2.0
152     mid_x = (points[:, 0].max() + points[:, 0].min()) * 0.5
153     mid_y = (points[:, 1].max() + points[:, 1].min()) * 0.5
154     mid_z = (points[:, 2].max() + points[:, 2].min()) * 0.5
155     ax.set_xlim(mid_x - max_range, mid_x + max_range)
156     ax.set_ylim(mid_y - max_range, mid_y + max_range)
157     ax.set_zlim(mid_z - max_range, mid_z + max_range)
158
159     plt.show()
160
161 valid = []
162
163 def simulate(t1=1.5, t2=3.6, t3=20):
164
165     ANS = 0
166
167     # 时间和速度
168     # 根据效率优化建议，将时间步长从0.001增加到0.05，可大幅提
169     # 升计算速度
170     INTERVAL = 0.01
171     V_M1 = 300.0
172     V_FY1 = 120.0
173     A_G_Z = -9.8
174
175     # 初始位置
176     P_M1_start = np.array([2000.0, 0.0, 2000.0])
177     P_FY1_start = np.array([17800.0, 0.0, 1800.0])
178
179     # 目标位置
180     TARGET_M1 = np.array([0.0, 0.0, 0.0])
181     TARGET_FY1 = np.array([0.0, 0.0, P_FY1_start[2]])
182
183     # 圆柱体参数

```

```

183 CYLINDER_CENTER_BASE = np.array([0.0, 200.0, 0.0])
184 CYLINDER_RADIUS = 7.0
185 CYLINDER_HEIGHT = 10.0
186
187 temp_alpha = 0.10
188 temp_alpha = np.radians(temp_alpha)
189
190 # 预计算单位方向向量
191 DIR_M1 = (TARGET_M1 - P_M1_start) / np.linalg.norm(
TARGET_M1 - P_M1_start)
192 #DIR_FY1 = np.array([-math.cos(temp_alpha), math.sin(
temp_alpha), 0.0]) # (0.9888, 0.1491, 0.0)
193 DIR_FY1 = np.array([-1.0, 0.0, 0.0])
194
195 # 预生成圆柱体表面采样点并进行可视化
196 cylinder_surface_points = generate_cylinder_points(
197     CYLINDER_CENTER_BASE, CYLINDER_RADIUS, CYLINDER_HEIGHT
198 )
199 #visualize_cylinder(cylinder_surface_points)
200
201 # --- 模拟开始 ---
202 # --- 第一阶段 ---
203 t_phase1_start = 0.0
204 t_phase1_end = t_phase1_start + t1
205 t_phase1_duration = t_phase1_end - t_phase1_start
206 pos_m1_p1 = P_M1_start + DIR_M1 * V_M1 * t_phase1_duration
207 pos_fy1_p1 = P_FY1_start + DIR_FY1 * V_FY1 *
t_phase1_duration
208
209 print(f"--- 第一阶段结束 (t = {t_phase1_duration}s) ---")
210 print(f"M1 坐标: {np.round(pos_m1_p1, 2)}")
211 print(f"FY1 坐标 (即G的投放点): {np.round(pos_fy1_p1, 2)}"
)
212 print("\n" + "=" * 40 + "\n")
213

```

```

214 # --- 第二阶段（干扰弹做抛体运动）---
215 t_phase2_start = t_phase1_end
216 t_phase2_end = t_phase2_start + t2
217 t_phase2_duration = t_phase2_end - t_phase2_start
218
219 # M1 和 FY1 在 t=5.1s 的位置
220 pos_m1_p2 = P_M1_start + DIR_M1 * V_M1 * t_phase2_end
221 pos_fy1_p2 = P_FY1_start + DIR_FY1 * V_FY1 * t_phase2_end
222
223 # 核心物理模型===
224 # 干扰弹(G)在投放后做抛体运动，其轨迹独立于无人机(FY1)。
225 # G的初始位置是在 t=1.5s 时无人机的位置
226 pos_g_launch = pos_fy1_p1
227 # G的初始速度继承自无人机（矢量）
228 vel_g_launch = DIR_FY1 * V_FY1
229 # 根据抛体运动公式计算G在第二阶段结束(t=5.1s)时的位置
230 # 水平位移：pos_xy = pos_start_xy + v_xy * t
231 pos_g_p2_xy = pos_g_launch[:2] + vel_g_launch[:2] *
t_phase2_duration
232 # 垂直位移：z = z0 + v0_z*t + 0.5*a*t^2（垂直初速度v0_z为0
）
233 pos_g_p2_z = pos_g_launch[2] + 0.5 * A_G_Z * (
t_phase2_duration ** 2)
234 # 组合成G的最终坐标，即烟幕起爆点
235 pos_g_p2 = np.array([pos_g_p2_xy[0], pos_g_p2_xy[1],
pos_g_p2_z])
236
237 print(f"--- 第二阶段结束 (t = {t2}s) ---")
238 print(f"M1 坐标: {np.round(pos_m1_p2, 2)}")
239 print(f"FY1 坐标: {np.round(pos_fy1_p2, 2)}")
240 print(f"G 坐标 (烟幕起爆点): {np.round(pos_g_p2, 2)}")
241 print("\n" + "=" * 40 + "\n")
242
243 # --- 第三阶段（烟幕云团下沉并进行遮蔽判断）---
244 t_phase3_start = t_phase2_end

```

```

245     t_phase3_end = t_phase3_start + t3
246     V_SMOKE_Z = -3.0 # 烟幕下沉速度
247
248     # 记录烟幕云团开始下沉的初始位置
249     pos_smoke_start = pos_g_p2
250
251     print(f"--- 第三阶段模拟开始 (t = {t_phase3_start}s to {
252         t_phase3_end}s, 步长={INTERVAL}s) ---")
253     for t in np.arange(t_phase3_start, t_phase3_end, INTERVAL)
254         :
255         # 计算当前时刻各物体位置
256         pos_m1_current = P_M1_start + DIR_M1 * V_M1 * t
257
258         t_since_detonation = t - t_phase3_start
259         pos_smoke_current = pos_smoke_start + np.array([0, 0,
260             V_SMOKE_Z * t_since_detonation])
261
262         # 执行遮蔽判定
263         is_obscured = True
264         for cyl_point in cylinder_surface_points:
265             dist = point_segment_distance(pos_smoke_current,
266                 pos_m1_current, cyl_point)
267             if dist > 10.0:
268                 is_obscured = False
269                 break
270
271         if is_obscured:
272             ANS += INTERVAL
273             valid.append(t)
274
275     print("\n" + "=" * 40 + "\n")
276     print(f"模拟结束。")
277     print(f"最终计算出的有效遮蔽总时长为: {ANS:.4f} 秒")
278     print(f'区间为: [{valid[0]:.2f}, {valid[-1]:.2f}]')
279     return ANS

```



```

276
277 def t1():
278     drop_times_to_test = np.arange(0, 5.25, 0.25)
279     obscuration_durations = []
280
281     print("开始进行参数敏感性分析...")
282     for i, t_d in enumerate(drop_times_to_test):
283         print(f"正在计算： 投放时间 = {t_d:.2f}s ({i + 1}/{len(drop_times_to_test)})")
284         duration = simulate(t1=float(t_d))
285         obscuration_durations.append(duration)
286
287     # 寻找最佳结果
288     max_duration = max(obscuration_durations)
289     best_t_drop = drop_times_to_test[np.argmax(obscuration_durations)]
290
291     print(f"最佳投放时间点： {best_t_drop:.2f}秒")
292     print(f"对应的最长有效遮蔽时长： {max_duration:.4f}秒")
293
294     # --- 结果可视化 ---
295     plt.figure(figsize=(12, 7))
296     plt.plot(drop_times_to_test, obscuration_durations, marker='o', linestyle='-', label='bottom')
297
298     # 突出显示最大值点
299     plt.plot(best_t_drop, max_duration, 'ro', markersize=10, label=f'best: {max_duration:.2f}s')
300     plt.annotate(f'({best_t_drop:.2f}s, {max_duration:.2f}s)',
301                 xy=(best_t_drop, max_duration),
302                 xytext=(best_t_drop + 0.3, max_duration - 0.1),
303                 )
304
305     plt.title('有效遮蔽时长 vs. 干扰弹投放时间', fontsize=16)

```

```

306     plt.xlabel('第一阶段飞行时间（投放时机 t_drop） / 秒',
fontsize=12)
307     plt.ylabel('有效遮蔽总时长 / 秒', fontsize=12)
308     plt.grid(True)
309     plt.legend()
310     plt.show()
311
312 if __name__ == "__main__":
313     simulate()
314     # visualize_cylinder(generate_cylinder_points(
315     #     np.array([0.0, 200.0, 0.0]), 7.0, 10.0, num_points
=10000
316     # ))

```

q2.py

```

1  import numpy as np
2  from shapely.geometry import Polygon, LineString, Point
3  import math
4  import matplotlib.pyplot as plt
5  from mpl_toolkits.mplot3d import Axes3D
6  import sys
7
8
9  class Logger(object):
10     def __init__(self, filename="record.txt"):
11         self.terminal = sys.stdout
12
13         self.log = open(filename, "w", encoding='utf-8')
14
15     def write(self, message):
16         self.terminal.write(message)
17         self.log.write(message)
18
19     def flush(self):
20

```

```

21         self.terminal.flush()
22         self.log.flush()
23
24 # 将标准输出重定向到我们自定义的Logger实例
25 sys.stdout = Logger("record.txt")
26
27 # --- 辅助函数定义 ---
28 def point_segment_distance(p, a, b):
29     ab = b - a
30     if np.dot(ab, ab) == 0: return np.linalg.norm(p - a)
31     ap = p - a
32     t = np.dot(ap, ab) / np.dot(ab, ab)
33     if t < 0.0:
34         return np.linalg.norm(p - a)
35     elif t > 1.0:
36         return np.linalg.norm(p - b)
37     else:
38         projection = a + t * ab
39         return np.linalg.norm(p - projection)
40
41
42 def generate_cylinder_points(center, radius, height,
43                               num_points):
44     points = []
45     num_side = int(num_points * 0.7)
46     for _ in range(num_side):
47         angle = np.random.uniform(0, 2 * np.pi)
48         z = center[2] + np.random.uniform(0, height)
49         x = center[0] + radius * np.cos(angle)
50         y = center[1] + radius * np.sin(angle)
51         points.append([x, y, z])
52     num_cap = (num_points - num_side) // 2
53     for _ in range(num_cap):
54         angle = np.random.uniform(0, 2 * np.pi)
55         r = np.sqrt(np.random.uniform(0, 1)) * radius

```

```

55     x_bot = center[0] + r * np.cos(angle)
56     y_bot = center[1] + r * np.sin(angle)
57     points.append([x_bot, y_bot, center[2]])
58     x_top = center[0] + r * np.cos(angle)
59     y_top = center[1] + r * np.sin(angle)
60     points.append([x_top, y_top, center[2] + height])
61     return np.array(points)
62
63
64 def calculate_ans_for_phase3(pos_g_start, T_start):
65     ans = 0.0
66     t_phase3_start = T_start
67     t_phase3_end = t_phase3_start + 20.0
68     for t in np.arange(t_phase3_start, t_phase3_end +
69 P3_INTERVAL / 2, P3_INTERVAL):
70         t_since_p3_start = t - t_phase3_start
71         pos_m1_current = P_M1_start + DIR_M1 * V_M1 * t
72         pos_g_current = np.copy(pos_g_start)
73         pos_g_current[2] -= V_G_Z_p3 * t_since_p3_start
74         all_ok = True
75         for cyl_point in cylinder_surface_points:
76             dist = point_segment_distance(pos_g_current,
77 pos_m1_current, cyl_point)
78             if dist > 10.0:
79                 all_ok = False
80                 break
81         if all_ok:
82             ans += P3_INTERVAL
83     return ans
84
85 V_INTERVAL = 5
86 T_INTERVAL = 1 # T的步长可以适当增大，因为优化算法会帮助我们
87                 找到Z
88
89 ANGLE_STEP_DEG = 1 # 角度步长

```

```

87 Z_STEP = 2 # Z坐标步长
88 P3_INTERVAL = 0.01
89 NUM_CYLINDER_POINTS = 100
90
91 # --- 全局常量 ---
92 V_M1 = 300.0
93 V_G_Z_p3 = 3.0
94 A_G_Z = -9.8
95 P_M1_start = np.array([20000.0, 0.0, 2000.0])
96 P_FY1_start = np.array([12000.0, 1400.0, 1400.0])
97 TARGET_M1 = np.array([0.0, 0.0, 0.0])
98 CYLINDER_CENTER = np.array([0.0, 200.0, 0.0])
99 DIR_M1 = (TARGET_M1 - P_M1_start) / np.linalg.norm(TARGET_M1 -
    P_M1_start)
100 cylinder_surface_points = generate_cylinder_points(
101     CYLINDER_CENTER, 7.0, 10.0, NUM_CYLINDER_POINTS
102 )
103
104 # --- 主搜索循环 ---
105 global_max_ans = -1
106 global_best_params = {}
107
108 v_min = 70
109 v_max = 140
110
111 for V_FY1 in np.arange(v_min, v_max + V_INTERVAL, V_INTERVAL):
112     print(f"\n{'=' * 20} Testing V_FY1 = {V_FY1:.1f} {'=' *
    20}")
113     T_min = 5
114     T_max = 25
115     # 用于记录上一个T的最优z
116     z_optimal_for_previous_T = None
117
118     for T in np.arange(T_min, T_max, T_INTERVAL):
119         pos_m1_T = P_M1_start + DIR_M1 * V_M1 * T

```

```

120         pos_m1_T_plus_20 = P_M1_start + DIR_M1 * V_M1 * (T +
121         20.0)
122         cir_center_xy = P_FY1_start[:2]
123         cir_radius = V_FY1 * T
124
125         v1 = pos_m1_T[:2]
126         v2 = pos_m1_T_plus_20[:2]
127         v3 = CYLINDER_CENTER[:2]
128         dist_to_edge1 = point_segment_distance(cir_center_xy,
129         v1, v2)
130         dist_to_edge2 = point_segment_distance(cir_center_xy,
131         v2, v3)
132         dist_to_edge3 = point_segment_distance(cir_center_xy,
133         v3, v1)
134         min_dist_to_triangle = min(dist_to_edge1,
135         dist_to_edge2, dist_to_edge3)
136         if min_dist_to_triangle > cir_radius:
137             continue
138
139         # --- 计算几何相交区域 ---
140         triangle = Polygon([v1, v2, v3])
141         circle_boundary = LineString(
142             [(cir_center_xy[0] + cir_radius * np.cos(a),
143             cir_center_xy[1] + cir_radius * np.sin(a)) for a in
144             np.linspace(0, 2 * np.pi, 360)])
145         intersection = triangle.intersection(circle_boundary)
146         if intersection.is_empty:
147             continue
148
149         # --- 确定G点Z坐标范围 ---
150         z_max = P_FY1_start[2]
151         z_min = P_FY1_start[2] + 0.5 * A_G_Z * (T ** 2)
152
153         # --- 5. 遍历候选点，寻找最大ANS ---
154         max_ans_for_this_T = -1

```

```

149         best_g_for_this_T = None
150
151         # 优化1: 自适应调整Z的搜索起点
152         if z_optimal_for_previous_T is not None:
153             z_start_guess = z_optimal_for_previous_T + 50.0
154             z_start = max(z_min, min(z_start_guess, z_max))
155         else:
156             z_start = z_max
157
158         candidate_arcs = [intersection] if not hasattr(
intersection, 'geoms') else intersection.geoms
159         for arc in candidate_arcs:
160             start_angle = math.atan2(arc.coords[0][1] -
cir_center_xy[1], arc.coords[0][0] - cir_center_xy[0])
161             end_angle = math.atan2(arc.coords[-1][1] -
cir_center_xy[1], arc.coords[-1][0] - cir_center_xy[0])
162             if end_angle < start_angle: end_angle += 2 * np.pi
163             num_angle_steps = int(np.rad2deg(abs(end_angle -
start_angle)) / ANGLE_STEP_DEG) + 1
164
165             for angle in np.linspace(start_angle, end_angle,
num_angle_steps):
166                 g_xy = np.array(
167                     [cir_center_xy[0] + cir_radius * np.cos(
angle), cir_center_xy[1] + cir_radius * np.sin(angle)])
168
169                 # 优化2: 为提前退出做准备
170                 prev_ans_1 = -1.0
171                 prev_ans_2 = -1.0
172                 best_z_for_this_angle = None
173                 max_ans_for_this_angle = -1
174
175                 for z in np.arange(z_min, z_start, Z_STEP)
[:: -1]:
176                     pos_g_start = np.array([g_xy[0], g_xy[1],

```

```

z])
177         ans = calculate_ans_for_phase3(pos_g_start
    , T)
178
179         if ans < prev_ans_1 < prev_ans_2:
180             break # 提前退出
181         prev_ans_2 = prev_ans_1
182         prev_ans_1 = ans
183
184         if ans > max_ans_for_this_angle:
185             max_ans_for_this_angle = ans
186             best_z_for_this_angle = z
187
188         if ans > max_ans_for_this_T:
189             max_ans_for_this_T = ans
190             best_g_for_this_T = pos_g_start
191             if ans > global_max_ans:
192                 global_max_ans = ans
193                 global_best_params = {'V_FY1':
V_FY1, 'T': T, 'P_G': pos_g_start}
194
195                 # 在T循环结束时，更新用于下一个T的
z_optimal
196
197         z_optimal_for_previous_T = best_g_for_this_T[2]
198         if best_g_for_this_T is not None:
199             print(
200                 f"V_FY1={V_FY1:.1f}, T={T:.1f}, Max ANS={
max_ans_for_this_T:.3f}, G_start=({best_g_for_this_T[0]:.2f
}, {best_g_for_this_T[1]:.2f}, {best_g_for_this_T[2]:.2f}))"
201
202 # --- 全局最优解分析与可视化 ---
203 print("\n" + "*" * 30)
204 print("全局最优解分析:")
205 if global_max_ans > -1:

```



```

206     V_FY1_opt, T_opt, P_G_opt = global_best_params['V_FY1'],
global_best_params['T'], global_best_params['P_G']
207     displacement_fy1_xy = P_G_opt[:2] - P_FY1_start[:2]
208     norm = np.linalg.norm(displacement_fy1_xy)
209     dir_fy1_opt = displacement_fy1_xy / norm if norm > 0 else
np.array([0.0, 0.0])
210     delta_z = P_G_opt[2] - P_FY1_start[2]
211     t2_opt = np.sqrt(2 * delta_z / A_G_Z) if delta_z <= 0 else
0
212     t1_opt = T_opt - t2_opt
213     P_M1_p3_start_opt = P_M1_start + DIR_M1 * V_M1 * T_opt
214
215     print(f"最大 ANS: {global_max_ans:.3f}")
216     print("\n达成此结果的四个核心变量为:")
217     print(f"  1. FY1 飞行速度: {V_FY1_opt:.2f} 单位/秒")
218     print(f"  2. FY1 飞行方向向量 (XY): ({dir_fy1_opt[0]:.4f},
{dir_fy1_opt[1]:.4f})")
219     print(f"  3. 第一阶段持续时间 (t1): {t1_opt:.3f} 秒")
220     print(f"  4. 第二阶段持续时间 (t2): {t2_opt:.3f} 秒")
221
222     print("\n正在生成最优解的关键点位置图...")
223     fig = plt.figure(figsize=(12, 9))
224     ax = fig.add_subplot(111, projection='3d')
225     ax.scatter(P_FY1_start[0], P_FY1_start[1], P_FY1_start[2],
c='blue', s=100, label='FY1 Start', marker='s')
226     ax.scatter(P_G_opt[0], P_G_opt[1], P_G_opt[2], c='green',
s=150, label='G Start (P3)', marker='*')
227     ax.scatter(P_M1_p3_start_opt[0], P_M1_p3_start_opt[1],
P_M1_p3_start_opt[2], c='red', s=100, label='M1 Start (P3)',
228                 marker='o')
229     ax.scatter(0, 0, 0, c='black', s=80, label='Origin',
marker='+')
230     ax.scatter(CYLINDER_CENTER[0], CYLINDER_CENTER[1],
CYLINDER_CENTER[2], c='gray', s=80, label='Cylinder Center',
231                 marker='D')

```

```

232     ax.text(P_FY1_start[0], P_FY1_start[1], P_FY1_start[2], '
FY1 Start', color='blue')
233     ax.text(P_G_opt[0], P_G_opt[1], P_G_opt[2], '  G Start (P3
)', color='green', fontsize=12)
234     ax.text(P_M1_p3_start_opt[0], P_M1_p3_start_opt[1],
P_M1_p3_start_opt[2], '  M1 Start (P3)', color='red')
235     ax.text(0, 0, 0, '  Origin', color='black')
236     ax.text(CYLINDER_CENTER[0], CYLINDER_CENTER[1],
CYLINDER_CENTER[2], '  Cylinder Center', color='gray')
237     ax.plot([P_FY1_start[0], P_G_opt[0]], [P_FY1_start[1],
P_G_opt[1]], [P_FY1_start[2], P_FY1_start[2]], 'b--',
238             alpha=0.5, label='FY1 Path (XY Projection)')
239     ax.plot([P_G_opt[0]], [P_G_opt[1]], [P_FY1_start[2],
P_G_opt[2]], 'g--', alpha=0.5, label='G Z-Drop')
240     ax.set_xlabel('X Coordinate')
241     ax.set_ylabel('Y Coordinate')
242     ax.set_zlabel('Z Coordinate')
243     ax.set_title('Key Positions for Global Optimal Solution')
244     ax.legend()
245     ax.invert_yaxis()
246     ax.view_init(elev=25, azim=-75)
247     plt.show()
248 else:
249     print("在搜索范围内未找到任何有效的 (ANS > 0) 解。")

```

q3.py

```

1  from typing import Any
2
3  import numpy as np
4  from shapely.geometry import Polygon, LineString, Point
5  import math
6  import matplotlib.pyplot as plt
7  from mpl_toolkits.mplot3d import Axes3D
8  import sys
9  from tqdm import tqdm

```

```

10
11 def point_segment_distance(p, a, b):
12     ab = b - a
13     if np.dot(ab, ab) == 0: return np.linalg.norm(p - a)
14     ap = p - a
15     t = np.dot(ap, ab) / np.dot(ab, ab)
16     if t < 0.0:
17         return np.linalg.norm(p - a)
18     elif t > 1.0:
19         return np.linalg.norm(p - b)
20     else:
21         projection = a + t * ab
22         return np.linalg.norm(p - projection)
23
24
25 def generate_cylinder_points(center, radius, height,
    num_points):
26     points = []
27     num_side = int(num_points * 0.7)
28     for _ in range(num_side):
29         angle = np.random.uniform(0, 2 * np.pi)
30         z = center[2] + np.random.uniform(0, height)
31         x = center[0] + radius * np.cos(angle)
32         y = center[1] + radius * np.sin(angle)
33         points.append([x, y, z])
34     num_cap = (num_points - num_side) // 2
35     for _ in range(num_cap):
36         angle = np.random.uniform(0, 2 * np.pi)
37         r = np.sqrt(np.random.uniform(0, 1)) * radius
38         x_bot = center[0] + r * np.cos(angle)
39         y_bot = center[1] + r * np.sin(angle)
40         points.append([x_bot, y_bot, center[2]])
41         x_top = center[0] + r * np.cos(angle)
42         y_top = center[1] + r * np.sin(angle)
43         points.append([x_top, y_top, center[2] + height])

```

```

44     return np.array(points)
45
46
47 def calculate_ans_for_phase3(t1=0.00714286, t2=0, t3=20,
    temp_alpha=0, V_FY1=140):
48
49     valid = []
50     ANS = 0
51
52     # 时间和速度
53     # 根据效率优化建议，将时间步长从0.001增加到0.05，可大幅提
    升计算速度
54     INTERVAL = 0.01
55     V_M1 = 300.0
56     A_G_Z = -9.8
57
58     # 初始位置
59     P_M1_start = np.array([2000.0, 0.0, 2000.0])
60     P_FY1_start = np.array([17800.0, 0.0, 1800.0])
61
62     # 目标位置
63     TARGET_M1 = np.array([0.0, 0.0, 0.0])
64     TARGET_FY1 = np.array([0.0, 0.0, P_FY1_start[2]])
65
66     # 圆柱体参数
67     CYLINDER_CENTER_BASE = np.array([0.0, 200.0, 0.0])
68     CYLINDER_RADIUS = 7.0
69     CYLINDER_HEIGHT = 10.0
70
71     temp_alpha = np.radians(temp_alpha)
72
73     # 预计算单位方向向量
74     DIR_M1 = (TARGET_M1 - P_M1_start) / np.linalg.norm(
    TARGET_M1 - P_M1_start)
75     DIR_FY1 = np.array([-math.cos(temp_alpha), math.sin(

```

```

temp_alpha), 0.0]) # (0.9888, 0.1491, 0.0)
76
77 # --- 模拟开始 ---
78 # --- 第一阶段 ---
79 t_phase1_start = 0.0
80 t_phase1_end = t_phase1_start + t1
81 t_phase1_duration = t_phase1_end - t_phase1_start
82 pos_m1_p1 = P_M1_start + DIR_M1 * V_M1 * t_phase1_duration
83 pos_fy1_p1 = P_FY1_start + DIR_FY1 * V_FY1 *
t_phase1_duration
84
85 # --- 第二阶段（干扰弹做抛体运动） ---
86 t_phase2_start = t_phase1_end
87 t_phase2_end = t_phase2_start + t2
88 t_phase2_duration = t_phase2_end - t_phase2_start
89
90 # M1 和 FY1 在 t=5.1s 的位置
91 pos_m1_p2 = P_M1_start + DIR_M1 * V_M1 * t_phase2_end
92 pos_fy1_p2 = P_FY1_start + DIR_FY1 * V_FY1 * t_phase2_end
93
94 # 干扰弹(G)在投放后做抛体运动，其轨迹独立于无人机(FY1)。
95 # G的初始位置是在 t=1.5s 时无人机的位置
96 pos_g_launch = pos_fy1_p1
97 # G的初始速度继承自无人机（矢量）
98 vel_g_launch = DIR_FY1 * V_FY1
99 # 根据抛体运动公式计算G在第二阶段结束(t=5.1s)时的位置
100 # 水平位移：pos_xy = pos_start_xy + v_xy * t
101 pos_g_p2_xy = pos_g_launch[:2] + vel_g_launch[:2] *
t_phase2_duration
102 # 垂直位移：z = z0 + v0_z*t + 0.5*a*t^2（垂直初速度v0_z为0
）
103 pos_g_p2_z = pos_g_launch[2] + 0.5 * A_G_Z * (
t_phase2_duration ** 2)
104 # 组合成G的最终坐标，即烟幕起爆点
105 pos_g_p2 = np.array([pos_g_p2_xy[0], pos_g_p2_xy[1],

```

```

pos_g_p2_z])
106
107     # --- 第三阶段（烟幕云团下沉并进行遮蔽判断） ---
108     t_phase3_start = t_phase2_end
109     t_phase3_end = t_phase3_start + t3
110     V_SMOKE_Z = -3.0 # 烟幕下沉速度
111
112     # 记录烟幕云团开始下沉的初始位置
113     pos_smoke_start = pos_g_p2
114
115     # print(f"--- 第三阶段模拟开始 (t = {t_phase3_start}s to {
116     t_phase3_end}s, 步长={INTERVAL}s) ---")
117     for t in np.arange(t_phase3_start, t_phase3_end, INTERVAL)
118     :
119         # 计算当前时刻各物体位置
120         pos_m1_current = P_M1_start + DIR_M1 * V_M1 * t
121
122         t_since_detonation = t - t_phase3_start
123         pos_smoke_current = pos_smoke_start + np.array([0, 0,
124         V_SMOKE_Z * t_since_detonation])
125
126         # 执行遮蔽判定
127         is_obscured = True
128         for cyl_point in cylinder_surface_points:
129             dist = point_segment_distance(pos_smoke_current,
130             pos_m1_current, cyl_point)
131             if dist > 10.0:
132                 is_obscured = False
133                 break
134
135         if is_obscured:
136             ANS += INTERVAL
137             valid.append(t)
138
139         elif len(valid) != 0:

```

```

136         break
137     if len(valid) == 0:
138         ans = -1
139     else:
140         ans = valid[-1]
141     return ANS, ans
142
143
144 mode = 0
145
146 V_INTERVAL = 10 if mode == 0 else 5
147 T_INTERVAL = 1 if mode == 0 else 0.1
148 ANGLE_STEP_DEG = 1
149 Z_STEP = 0.5
150 P3_INTERVAL = 0.1 if mode == 0 else 0.005
151 NUM_CYLINDER_POINTS = 100 if mode == 0 else 500
152
153 V_M1 = 300.0
154 V_G_Z_p3 = 3.0
155 A_G_Z = -9.8
156 P_M1_start = np.array([20000.0, 0.0, 2000.0])
157 P_FY1_start = np.array([17800.0, 0.0, 1800.0])
158 TARGET_M1 = np.array([0.0, 0.0, 0.0])
159 CYLINDER_CENTER = np.array([0.0, 200.0, 0.0])
160 DIR_M1 = (TARGET_M1 - P_M1_start) / np.linalg.norm(TARGET_M1 -
    P_M1_start)
161 cylinder_surface_points = generate_cylinder_points(
162     CYLINDER_CENTER, 7.0, 10.0, NUM_CYLINDER_POINTS
163 )
164
165 global_max_ans = -1
166 global_best_params = []
167
168 v_min = 70
169 v_max = 140

```

```

170
171
172 def function1(x: int, V: int, alpha: float):
173     return (3 * x) / V + 20 + (math.cos(alpha) / P_FY1_start
174         [0]) * (P_FY1_start[2] - 20) * x + \
175         (3 / (math.cos(math.atan2(P_M1_start[2], P_M1_start
176         [0]))) * V_M1)) * \
177         (CYLINDER_CENTER[0] + (CYLINDER_CENTER[1] / (
178         CYLINDER_CENTER[1] - x * math.sin(alpha)))) * (
179         P_FY1_start[0] - x * math.cos(alpha) -
180         CYLINDER_CENTER[0]) - P_M1_start[0])
181
182
183 def function2(x: int, V: int):
184     return (9.8 / (2 * V * V)) * x * x
185
186
187 def function3(x: int, alpha: float):
188     return (math.cos(np.radians(alpha)) * x * (P_FY1_start[2]
189     - 20)) / P_FY1_start[0] + 20
190
191
192 def function4(x: int, alpha: float):
193     return math.cos(alpha) / P_FY1_start[0] * (P_FY1_start[2]
194     - 20) * x + 20
195
196
197 def plot_heatmap(alpha_range, v_range, results_grid):
198     """
199     绘制二维颜色矩阵（热力图）
200     """
201     print("\n" + "*" * 30)
202     print("正在生成最终结果的热力图...")
203
204     plt.figure(figsize=(10, 8))

```



```

199     im = plt.imshow(results_grid.T, origin='lower', aspect='
200         extent=[alpha_range.min(), alpha_range.max
201         (), v_range.start, v_range.stop],
202         cmap='viridis')
203
204     cbar = plt.colorbar()
205     cbar.set_label('Max ANS (ans1+ans2+ans3)')
206
207     plt.title('Max ANS Heatmap for Angle vs. Speed', fontsize
208     =16)
209     plt.xlabel('Angle (degrees)', fontsize=12)
210     plt.ylabel('Speed (V_FY1)', fontsize=12)
211
212     plt.show()
213
214 def check(a: float, b: float, tmp: float, v: float, alpha:
215 float) -> bool:
216     """
217     根据给定的五个参数，计算M1和G点的瞬时位置，并进行成功判
218     定。
219
220     参数：
221     a (float): 用于计算G点位置的参数，可理解为水平位移相关
222     量。
223     b (float): 用于计算G点位置的参数，可理解为初始Z坐标下
224     降量。
225     tmp (float): 绝对时间戳 (t)。
226     v (float): FY系列飞机的速度 (V_FY)。
227     alpha (float): FY系列飞机的飞行角度 (角度制)。
228
229     返回：
230     bool: 如果判定为 "yes", 返回 True; 否则返回 False。
231     """

```

```

227
228     alpha_rad = math.radians(alpha)
229
230     T = a / v
231     t2 = math.sqrt(2 * b / 9.8)
232     t1 = T - t2
233     g_x, g_y, g_z = P_FY1_start[0], P_FY1_start[1],
P_FY1_start[2]
234     if tmp < t1:
235         g_x -= v * tmp * math.cos(alpha_rad)
236         g_y += v * tmp * math.sin(alpha_rad)
237     elif tmp < T:
238         g_x -= v * tmp * math.cos(alpha_rad)
239         g_y += v * tmp * math.sin(alpha_rad)
240         g_z -= 9.8 * (tmp - t1) * (tmp - t1) / 2
241     else:
242         # print("!")
243         g_x -= v * T * math.cos(alpha_rad)
244         g_y += v * T * math.sin(alpha_rad)
245         g_z -= 9.8 * t2 * t2 / 2 + 3 * (tmp - T)
246     # print(g_x)
247     # print(g_y)
248     # print(g_z)
249     pos_g_current = np.array([g_x, g_y, g_z])
250
251     # 3. 计算M1在 tmp 时刻的坐标
252     pos_m1_current = P_M1_start + DIR_M1 * V_M1 * tmp
253
254     # 4. 执行判定逻辑
255     for cyl_point in cylinder_surface_points:
256         dist = point_segment_distance(pos_g_current,
pos_m1_current, cyl_point)
257         # 如果任何一个距离大于等于10, 则判定失败, 立即返回
False
258         if dist >= 10.0:

```

```

259         return False
260
261     # 如果循环正常结束, 说明所有点都满足条件, 判定成功, 返回
    True
262     return True
263
264
265 def search():
266     global global_max_ans, global_best_params,
    global_best_alpha, global_best_V
267     ANGLE_INTERVAL = 0.1 if mode == 0 else 0.01
268     buffer = 0.1
269
270     alpha_range_deg = np.arange(0.0, 1, ANGLE_INTERVAL)
271     v_range = range(v_min, v_max + 1, V_INTERVAL)
272     # 创建一个网格来存储每个(alpha, V)组合的最大ANS (mx)
273     results_grid = np.zeros((len(alpha_range_deg), len(v_range)
    )))
274
275     for i, alpha in enumerate(tqdm(alpha_range_deg, desc="
    Angle Progress")):
276         alpha_rad = np.radians(alpha)
277         for j, V in enumerate(v_range):
278             mx = -1
279             best = []
280             for x1 in np.arange(0.0, 100, 1):
281                 y1 = function1(x1, V, alpha_rad)
282                 y1 = max(y1, 0)
283                 y2 = function2(x1, V)
284                 if y1 > y2:
285                     break
286                 y = min(y1, y2)
287                 p_g = np.array(
288                     [P_FY1_start[0] - x1 * math.cos(alpha_rad)
    , x1 * math.sin(alpha_rad), P_FY1_start[2] - y])

```

```

289         if V == 0:
290             t = 0
291         else:
292             t = x1 / V
293             t1 = x1 / V - math.sqrt(2 * y / 9.8)
294             ans1, tmp = calculate_ans_for_phase3(t1, t -
t1, 20, alpha, V)
295
296             print(f"alpha={alpha:.1f}°, V_FY1={V:.1f},
ANS1={ans1:.3f}, x1={x1:.2f},tmp = {tmp:.2f}")
297             x2_l, x2_r = x1, 1500
298             while x2_r - x2_l > 0.1:
299                 mid = (x2_l + x2_r) / 2
300                 y_mid = function1(mid, V, alpha_rad)
301                 if mid / V - math.sqrt(2 * y_mid / 9.8) <
t1 + 1 or tmp - (
302                     function3(mid, alpha) - y_mid) / 3
> mid / V or check(mid, y_mid, tmp, V, alpha):
303                     x2_l = mid
304                 else:
305                     x2_r = mid
306             x2, y2 = x2_r + 5, function1(x2_r + 5, V,
alpha_rad)
307             p_g_2 = np.array(
308                 [P_FY1_start[0] - x2 * math.cos(alpha_rad)
, x2 * math.sin(alpha_rad), P_FY1_start[2] - y2])
309             t, t2 = x2 / V, x2 / V - math.sqrt(2 * y2 /
9.8)
310
311             ans2, tmp = calculate_ans_for_phase3(t2, t -
t2, 20, alpha, V)
312             print(f"alpha={alpha:.1f}°, V_FY1={V:.1f},
ANS2={ans2:.3f}, x2={x2:.2f}")
313             x3_l, x3_r = x2, 2000
314             while x3_r - x3_l > 0.1:

```

```

315         mid = (x3_r + x3_l) / 2
316         y_mid = function1(mid, V, alpha_rad)
317         if mid / V - math.sqrt(2 * y_mid / 9.8) <
t2 + 1 or tmp - (
318             function3(mid, alpha) - y_mid) / 3
> mid / V or check(mid, y_mid, tmp, V, alpha):
319             x3_l = mid
320         else:
321             x3_r = mid
322             x3, y3 = x3_r + 5, function1(x3_r + 5, V,
alpha_rad)
323
324             p_g_3 = np.array(
325                 [P_FY1_start[0] - x3 * math.cos(alpha_rad)
, x3 * math.sin(alpha_rad), P_FY1_start[2] - y3])
326             t, t3 = x3 / V, x3 / V - math.sqrt(2 * y3 /
9.8)
327
328             ans3, _ = calculate_ans_for_phase3(t3, t - t3,
20, alpha, V)
329             print(f"alpha={alpha:.1f}°, V_FY1={V:.1f},
ANS3={ans3:.3f}, x3={x3:.2f}")
330             if ans1 + ans2 + ans3 > mx:
331                 mx = ans1 + ans2 + ans3
332                 best.clear()
333                 best.append([x1, y, x2, y2, x3, y3, ans1,
ans2, ans3])
334
335             print(f"ALL:alpha={alpha:.3f}°, V_FY1={V:.1f}, max
={mx:.3f}, best={best}")
336
337             if mx > -1:
338                 results_grid[i, j] = mx
339
340             if mx > global_max_ans:

```

```

341         global_max_ans = mx
342         global_best_params = best
343         global_best_alpha = alpha
344         global_best_V = V
345
346     print(
347         f"GLOBAL:bestalpha={global_best_alpha:.3f}°, bestV_FY1
={global_best_V:.1f}, ANS={global_max_ans:.3f}, params={
global_best_params}")
348
349     plot_heatmap(alpha_range_deg, v_range, results_grid)
350
351
352 def draw():
353     x_start = 0
354     x_end = 600
355
356     num_points = 4000
357
358
359     # a. 在指定的范围内，均匀地生成x坐标点
360     x_values = np.linspace(x_start, x_end, num_points)
361
362     # b. 分别计算两个函数对应的y值
363     y1_values = function1(x_values, 100, 0)
364     y2_values = function2(x_values, 100)
365     y3_values = function3(x_values, 0)
366
367     x_values = [-x for x in x_values]
368     y1_values = [-y for y in y1_values]
369     y2_values = [-y for y in y2_values]
370     y3_values = [-y for y in y3_values]
371
372     # c. 开始绘图
373     print("正在生成函数图像...")

```

```

374
375 plt.style.use('seaborn-v0_8-whitegrid')
376
377 fig, ax = plt.subplots(figsize=(12, 8))
378
379 ax.axhline(0, color='black', linewidth=0.7, zorder=0)
380 ax.axvline(0, color='black', linewidth=0.7, zorder=0)
381
382 ax.plot(x_values, y1_values, label='L1', color='#3498db',
linewidth=2.5)
383 ax.plot(x_values, y2_values, label='L2', color='#e74c3c',
linewidth=2.5)
384 ax.plot(x_values, y3_values, label='L3', color='#2ecc71',
linewidth=2.5)
385
386
387 ax.set_title('profile map', fontsize=18, fontweight='bold'
, pad=15)
388 ax.set_xlabel('x ', fontsize=14, labelpad=10)
389 ax.set_ylabel('y ', fontsize=14, labelpad=10)
390
391 # 动态调整y轴范围以适应两个函数，并增加10%的边距
392 y_min = min(np.min(y1_values), np.min(y2_values))
393 y_max = max(np.max(y1_values), np.max(y2_values))
394 y_range = y_max - y_min
395 ax.set_ylim(y_min - 0.1 * y_range, y_max + 0.1 * y_range)
396
397 ax.legend(fontsize=12, frameon=True, facecolor='white',
framealpha=0.9, shadow=True)
398
399 ax.grid(True, which='both', linestyle='--', linewidth=0.5)
400
401 plt.tight_layout()
402 plt.show()
403

```

```

404     print("图像已生成并显示。")
405
406
407 if __name__ == "__main__":
408     search()
409     # draw()

```

q4.py

```

1  import numpy as np
2  from shapely.geometry import Polygon, LineString, Point
3  import math
4  import matplotlib.pyplot as plt
5  from mpl_toolkits.mplot3d import Axes3D
6  import sys
7
8
9  class Logger(object):
10     def __init__(self, filename="record.txt"):
11         self.terminal = sys.stdout
12
13         self.log = open(filename, "w", encoding='utf-8')
14
15     def write(self, message):
16         self.terminal.write(message)
17         self.log.write(message)
18
19     def flush(self):
20
21         self.terminal.flush()
22         self.log.flush()
23
24 # 将标准输出重定向到我们自定义的Logger实例
25 sys.stdout = Logger("record.txt")
26
27 # --- 辅助函数定义 ---

```



```

28 def point_segment_distance(p, a, b):
29     ab = b - a
30     if np.dot(ab, ab) == 0: return np.linalg.norm(p - a)
31     ap = p - a
32     t = np.dot(ap, ab) / np.dot(ab, ab)
33     if t < 0.0:
34         return np.linalg.norm(p - a)
35     elif t > 1.0:
36         return np.linalg.norm(p - b)
37     else:
38         projection = a + t * ab
39         return np.linalg.norm(p - projection)
40
41
42 def generate_cylinder_points(center, radius, height,
    num_points):
43     points = []
44     num_side = int(num_points * 0.7)
45     for _ in range(num_side):
46         angle = np.random.uniform(0, 2 * np.pi)
47         z = center[2] + np.random.uniform(0, height)
48         x = center[0] + radius * np.cos(angle)
49         y = center[1] + radius * np.sin(angle)
50         points.append([x, y, z])
51     num_cap = (num_points - num_side) // 2
52     for _ in range(num_cap):
53         angle = np.random.uniform(0, 2 * np.pi)
54         r = np.sqrt(np.random.uniform(0, 1)) * radius
55         x_bot = center[0] + r * np.cos(angle)
56         y_bot = center[1] + r * np.sin(angle)
57         points.append([x_bot, y_bot, center[2]])
58         x_top = center[0] + r * np.cos(angle)
59         y_top = center[1] + r * np.sin(angle)
60         points.append([x_top, y_top, center[2] + height])
61     return np.array(points)

```

```

62
63
64 def calculate_ans_for_phase3(pos_g_start, T_start):
65     ans = 0.0
66     t_phase3_start = T_start
67     t_phase3_end = t_phase3_start + 20.0
68     for t in np.arange(t_phase3_start, t_phase3_end +
        P3_INTERVAL / 2, P3_INTERVAL):
69         t_since_p3_start = t - t_phase3_start
70         pos_m1_current = P_M1_start + DIR_M1 * V_M1 * t
71         pos_g_current = np.copy(pos_g_start)
72         pos_g_current[2] -= V_G_Z_p3 * t_since_p3_start
73         all_ok = True
74         for cyl_point in cylinder_surface_points:
75             dist = point_segment_distance(pos_g_current,
pos_m1_current, cyl_point)
76             if dist > 10.0:
77                 all_ok = False
78                 break
79         if all_ok:
80             ans += P3_INTERVAL
81     return ans
82
83
84 V_INTERVAL = 5
85 T_INTERVAL = 1 # T的步长可以适当增大，因为优化算法会帮助我们
    找到Z
86 ANGLE_STEP_DEG = 1 # 角度步长
87 Z_STEP = 2 # Z坐标步长
88 P3_INTERVAL = 0.01
89 NUM_CYLINDER_POINTS = 100
90
91 # --- 全局常量 ---
92 V_M1 = 300.0
93 V_G_Z_p3 = 3.0

```

```

94 A_G_Z = -9.8
95 P_M1_start = np.array([20000.0, 0.0, 2000.0])
96 P_FY1_start = np.array([12000.0, 1400.0, 1400.0])
97 TARGET_M1 = np.array([0.0, 0.0, 0.0])
98 CYLINDER_CENTER = np.array([0.0, 200.0, 0.0])
99 DIR_M1 = (TARGET_M1 - P_M1_start) / np.linalg.norm(TARGET_M1 -
    P_M1_start)
100 cylinder_surface_points = generate_cylinder_points(
101     CYLINDER_CENTER, 7.0, 10.0, NUM_CYLINDER_POINTS
102 )
103
104 # --- 主搜索循环 ---
105 global_max_ans = -1
106 global_best_params = {}
107
108 v_min = 70
109 v_max = 140
110
111 for V_FY1 in np.arange(v_min, v_max + V_INTERVAL, V_INTERVAL):
112     print(f"\n{'=' * 20} Testing V_FY1 = {V_FY1:.1f} {'=' *
113         20}")
114     T_min = 5
115     T_max = 25
116     # 用于记录上一个T的最优z
117     z_optimal_for_previous_T = None
118
119     for T in np.arange(T_min, T_max, T_INTERVAL):
120         pos_m1_T = P_M1_start + DIR_M1 * V_M1 * T
121         pos_m1_T_plus_20 = P_M1_start + DIR_M1 * V_M1 * (T +
122             20.0)
123
124         cir_center_xy = P_FY1_start[:2]
125         cir_radius = V_FY1 * T

```

```

126         v3 = CYLINDER_CENTER[:2]
127         dist_to_edge1 = point_segment_distance(cir_center_xy,
128         v1, v2)
129         dist_to_edge2 = point_segment_distance(cir_center_xy,
130         v2, v3)
131         dist_to_edge3 = point_segment_distance(cir_center_xy,
132         v3, v1)
133         min_dist_to_triangle = min(dist_to_edge1,
134         dist_to_edge2, dist_to_edge3)
135         if min_dist_to_triangle > cir_radius:
136             continue
137
138         # --- 计算几何相交区域 ---
139         triangle = Polygon([v1, v2, v3])
140         circle_boundary = LineString(
141             [(cir_center_xy[0] + cir_radius * np.cos(a),
142             cir_center_xy[1] + cir_radius * np.sin(a)) for a in
143             np.linspace(0, 2 * np.pi, 360)])
144         intersection = triangle.intersection(circle_boundary)
145         if intersection.is_empty:
146             continue
147
148         # --- 确定G点Z坐标范围 ---
149         z_max = P_FY1_start[2]
150         z_min = P_FY1_start[2] + 0.5 * A_G_Z * (T ** 2)
151
152         # --- 5. 遍历候选点，寻找最大ANS ---
153         max_ans_for_this_T = -1
154         best_g_for_this_T = None
155
156         # 优化1：自适应调整Z的搜索起点
157         if z_optimal_for_previous_T is not None:
158             z_start_guess = z_optimal_for_previous_T + 50.0
159             z_start = max(z_min, min(z_start_guess, z_max))
160         else:

```

```

156         z_start = z_max
157
158         candidate_arcs = [intersection] if not hasattr(
intersection, 'geoms') else intersection.geoms
159         for arc in candidate_arcs:
160             start_angle = math.atan2(arc.coords[0][1] -
cir_center_xy[1], arc.coords[0][0] - cir_center_xy[0])
161             end_angle = math.atan2(arc.coords[-1][1] -
cir_center_xy[1], arc.coords[-1][0] - cir_center_xy[0])
162             if end_angle < start_angle: end_angle += 2 * np.pi
163             num_angle_steps = int(np.rad2deg(abs(end_angle -
start_angle)) / ANGLE_STEP_DEG) + 1
164
165             for angle in np.linspace(start_angle, end_angle,
num_angle_steps):
166                 g_xy = np.array(
167                     [cir_center_xy[0] + cir_radius * np.cos(
angle), cir_center_xy[1] + cir_radius * np.sin(angle)])
168
169                 # 优化2: 为提前退出做准备
170                 prev_ans_1 = -1.0
171                 prev_ans_2 = -1.0
172                 best_z_for_this_angle = None
173                 max_ans_for_this_angle = -1
174
175                 for z in np.arange(z_min, z_start, Z_STEP)
[:: -1]:
176                     pos_g_start = np.array([g_xy[0], g_xy[1],
z])
177                     ans = calculate_ans_for_phase3(pos_g_start
, T)
178
179                     if ans < prev_ans_1 < prev_ans_2:
180                         break # 提前退出
181                     prev_ans_2 = prev_ans_1

```

```

182         prev_ans_1 = ans
183
184         if ans > max_ans_for_this_angle:
185             max_ans_for_this_angle = ans
186             best_z_for_this_angle = z
187
188         if ans > max_ans_for_this_T:
189             max_ans_for_this_T = ans
190             best_g_for_this_T = pos_g_start
191             if ans > global_max_ans:
192                 global_max_ans = ans
193                 global_best_params = {'V_FY1':
V_FY1, 'T': T, 'P_G': pos_g_start}
194
195                 # 在T循环结束时，更新用于下一个T的
z_optimal
196
197             z_optimal_for_previous_T = best_g_for_this_T[2]
198             if best_g_for_this_T is not None:
199                 print(
200                     f"V_FY1={V_FY1:.1f}, T={T:.1f}, Max ANS={
max_ans_for_this_T:.3f}, G_start=({best_g_for_this_T[0]:.2f
}, {best_g_for_this_T[1]:.2f}, {best_g_for_this_T[2]:.2f}))"
201
202 # --- 全局最优解分析与可视化 ---
203 print("\n" + "*" * 30)
204 print("全局最优解分析:")
205 if global_max_ans > -1:
206     V_FY1_opt, T_opt, P_G_opt = global_best_params['V_FY1'],
global_best_params['T'], global_best_params['P_G']
207     displacement_fy1_xy = P_G_opt[:2] - P_FY1_start[:2]
208     norm = np.linalg.norm(displacement_fy1_xy)
209     dir_fy1_opt = displacement_fy1_xy / norm if norm > 0 else
np.array([0.0, 0.0])
210     delta_z = P_G_opt[2] - P_FY1_start[2]

```

```

211     t2_opt = np.sqrt(2 * delta_z / A_G_Z) if delta_z <= 0 else
212     0
213     t1_opt = T_opt - t2_opt
214     P_M1_p3_start_opt = P_M1_start + DIR_M1 * V_M1 * T_opt
215
216     print(f"最大 ANS: {global_max_ans:.3f}")
217     print("\n达成此结果的四个核心变量为:")
218     print(f"  1. FY1 飞行速度: {V_FY1_opt:.2f} 单位/秒")
219     print(f"  2. FY1 飞行方向向量 (XY): ({dir_fy1_opt[0]:.4f},
220     {dir_fy1_opt[1]:.4f})")
221     print(f"  3. 第一阶段持续时间 (t1): {t1_opt:.3f} 秒")
222     print(f"  4. 第二阶段持续时间 (t2): {t2_opt:.3f} 秒")
223
224     print("\n正在生成最优解的关键点位置图...")
225     fig = plt.figure(figsize=(12, 9))
226     ax = fig.add_subplot(111, projection='3d')
227     ax.scatter(P_FY1_start[0], P_FY1_start[1], P_FY1_start[2],
228     c='blue', s=100, label='FY1 Start', marker='s')
229     ax.scatter(P_G_opt[0], P_G_opt[1], P_G_opt[2], c='green',
230     s=150, label='G Start (P3)', marker='*')
231     ax.scatter(P_M1_p3_start_opt[0], P_M1_p3_start_opt[1],
232     P_M1_p3_start_opt[2], c='red', s=100, label='M1 Start (P3)',
233     marker='o')
234     ax.scatter(0, 0, 0, c='black', s=80, label='Origin',
235     marker='+')
236     ax.scatter(CYLINDER_CENTER[0], CYLINDER_CENTER[1],
237     CYLINDER_CENTER[2], c='gray', s=80, label='Cylinder Center',
238     marker='D')
239     ax.text(P_FY1_start[0], P_FY1_start[1], P_FY1_start[2], '
240     FY1 Start', color='blue')
241     ax.text(P_G_opt[0], P_G_opt[1], P_G_opt[2], '  G Start (P3
242     )', color='green', fontsize=12)
243     ax.text(P_M1_p3_start_opt[0], P_M1_p3_start_opt[1],
244     P_M1_p3_start_opt[2], '  M1 Start (P3)', color='red')
245     ax.text(0, 0, 0, '  Origin', color='black')

```

```

236     ax.text(CYLINDER_CENTER[0], CYLINDER_CENTER[1],
CYLINDER_CENTER[2], 'Cylinder Center', color='gray')
237     ax.plot([P_FY1_start[0], P_G_opt[0]], [P_FY1_start[1],
P_G_opt[1]], [P_FY1_start[2], P_FY1_start[2]], 'b--',
238             alpha=0.5, label='FY1 Path (XY Projection)')
239     ax.plot([P_G_opt[0]], [P_G_opt[1]], [P_FY1_start[2],
P_G_opt[2]], 'g--', alpha=0.5, label='G Z-Drop')
240     ax.set_xlabel('X Coordinate')
241     ax.set_ylabel('Y Coordinate')
242     ax.set_zlabel('Z Coordinate')
243     ax.set_title('Key Positions for Global Optimal Solution')
244     ax.legend()
245     ax.invert_yaxis()
246     ax.view_init(elev=25, azim=-75)
247     plt.show()
248 else:
249     print("在搜索范围内未找到任何有效的 (ANS > 0) 解。")

```

q5.py

```

1  import numpy as np
2  import math
3  import matplotlib.pyplot as plt
4  from tqdm import tqdm
5
6
7  def point_segment_distance(p, a, b):
8      ab = b - a
9      if np.dot(ab, ab) == 0: return np.linalg.norm(p - a)
10     ap = p - a
11     t = np.dot(ap, ab) / np.dot(ab, ab)
12     if t < 0.0:
13         return np.linalg.norm(p - a)
14     elif t > 1.0:
15         return np.linalg.norm(p - b)
16     else:

```



```

17         projection = a + t * ab
18         return np.linalg.norm(p - projection)
19
20
21 def generate_cylinder_points(center, radius, height,
    num_points):
22     points = []
23     num_side = int(num_points * 0.7)
24     for _ in range(num_side):
25         angle = np.random.uniform(0, 2 * np.pi)
26         z = center[2] + np.random.uniform(0, height)
27         x = center[0] + radius * np.cos(angle)
28         y = center[1] + radius * np.sin(angle)
29         points.append([x, y, z])
30     num_cap = (num_points - num_side) // 2
31     for _ in range(num_cap):
32         angle = np.random.uniform(0, 2 * np.pi)
33         r = np.sqrt(np.random.uniform(0, 1)) * radius
34         x_bot = center[0] + r * np.cos(angle)
35         y_bot = center[1] + r * np.sin(angle)
36         points.append([x_bot, y_bot, center[2]])
37         x_top = center[0] + r * np.cos(angle)
38         y_top = center[1] + r * np.sin(angle)
39         points.append([x_top, y_top, center[2] + height])
40     return np.array(points)
41
42 def func(x, v, alpha, xm, ym, zm, xf, yf, zf):
43
44     try:
45         cos_alpha = math.cos(alpha)
46         sin_alpha = math.sin(alpha)
47         if cos_alpha == 0: return None
48
49         t = xf - x * cos_alpha
50         vm = 300.0

```

```

51
52     denominator = ym * t / xm - (sin_alpha * (xf - t) /
cos_alpha + yf - 200)
53     if denominator == 0: return None
54
55     k = 200.0 / denominator
56     xm1 = k * t
57     ym1 = 200.0 + k * (yf - 200.0 + sin_alpha * (xf - t) /
cos_alpha)
58     zm1 = k * zm * t / xm
59
60     dis_squared = (xm - xm1) ** 2 + (ym - ym1) ** 2 + (zm
- zm1) ** 2
61     if dis_squared < 0: return None
62     dis = math.sqrt(dis_squared)
63
64     result = 3 * x / v + zf - t * zm / xm - 3 * dis / vm
65     return result
66 except (ValueError, ZeroDivisionError):
67     return None
68
69 def calculate_ans_for_phase3(t1=0.00714286, t2=0, t3=20,
temp_alpha=1, V_FY1=1, P_M1_start = [], P_FY1_start=[]):
70
71     ANS = 0
72     valid = []
73
74     INTERVAL = 0.01
75     V_M1 = 300.0
76     A_G_Z = -9.8
77
78     # 目标位置
79     TARGET_M1 = np.array([0.0, 0.0, 0.0])
80     TARGET_FY1 = np.array([0.0, 0.0, P_FY1_start[2]])
81

```

```

82     # 圆柱体参数
83     CYLINDER_CENTER_BASE = np.array([0.0, 200.0, 0.0])
84     CYLINDER_RADIUS = 7.0
85     CYLINDER_HEIGHT = 10.0
86
87     temp_alpha = np.radians(temp_alpha)
88
89     # 预计算单位方向向量
90     DIR_M1 = (TARGET_M1 - P_M1_start) / np.linalg.norm(
91     TARGET_M1 - P_M1_start)
92     DIR_FY1 = np.array([-math.cos(temp_alpha), math.sin(
93     temp_alpha), 0.0]) # (0.9888, 0.1491, 0.0)
94     #visualize_cylinder(cylinder_surface_points)
95     # --- 第一阶段 ---
96     t_phase1_start = 0.0
97     t_phase1_end = t_phase1_start + t1
98     t_phase1_duration = t_phase1_end - t_phase1_start
99     pos_m1_p1 = P_M1_start + DIR_M1 * V_M1 * t_phase1_duration
100     pos_fy1_p1 = P_FY1_start + DIR_FY1 * V_FY1 *
101     t_phase1_duration
102
103     # --- 第二阶段（干扰弹做抛体运动） ---
104     t_phase2_start = t_phase1_end
105     t_phase2_end = t_phase2_start + t2
106     t_phase2_duration = t_phase2_end - t_phase2_start
107
108     # M1 和 FY1 在 t=5.1s 的位置
109     pos_m1_p2 = P_M1_start + DIR_M1 * V_M1 * t_phase2_end
110     pos_fy1_p2 = P_FY1_start + DIR_FY1 * V_FY1 * t_phase2_end
111
112     pos_g_launch = pos_fy1_p1
113     # G的初始速度继承自无人机（矢量）
114     vel_g_launch = DIR_FY1 * V_FY1
115     # 根据抛体运动公式计算G在第二阶段结束(t=5.1s)时的位置
116     # 水平位移: pos_xy = pos_start_xy + v_xy * t

```

```

114     pos_g_p2_xy = pos_g_launch[:2] + vel_g_launch[:2] *
t_phase2_duration
115     # 垂直位移:  $z = z_0 + v_{0z}t + 0.5at^2$  (垂直初速度 $v_{0z}$ 为0
)
116     pos_g_p2_z = pos_g_launch[2] + 0.5 * A_G_Z * (
t_phase2_duration ** 2)
117     # 组合成G的最终坐标, 即烟幕起爆点
118     pos_g_p2 = np.array([pos_g_p2_xy[0], pos_g_p2_xy[1],
pos_g_p2_z])
119     # --- 第三阶段 (烟幕云团下沉并进行遮蔽判断) ---
120     t_phase3_start = t_phase2_end
121     t_phase3_end = t_phase3_start + t3
122     V_SMOKE_Z = -3.0 # 烟幕下沉速度
123
124     # 记录烟幕云团开始下沉的初始位置
125     pos_smoke_start = pos_g_p2
126     for t in np.arange(t_phase3_start, t_phase3_end, INTERVAL)
:
127         # 计算当前时刻各物体位置
128         pos_m1_current = P_M1_start + DIR_M1 * V_M1 * t
129
130         t_since_detonation = t - t_phase3_start
131         pos_smoke_current = pos_smoke_start + np.array([0, 0,
V_SMOKE_Z * t_since_detonation])
132
133         # 执行遮蔽判定
134         is_obscured = True
135         for cyl_point in cylinder_surface_points:
136             dist = point_segment_distance(pos_smoke_current,
pos_m1_current, cyl_point)
137             if dist > 10.0:
138                 is_obscured = False
139                 break
140
141         if is_obscured:

```

```

142         ANS += INTERVAL
143         valid.append(t)
144     return ANS
145
146
147 # --- 全局常量和参数 ---
148 MODE = 0
149 V_INTERVAL = 10 if MODE == 0 else 5
150 ANGLE_INTERVAL = 10 if MODE == 0 else 0.5
151 X_INTERVAL = 50 if MODE == 0 else 20
152 NUM_CYLINDER_POINTS = 100
153 V_M1 = 300.0
154 A_G_Z = -9.8
155 P_M1_start = np.array([20000.0, 0.0, 2000.0])
156 P_M2_start = np.array([19000.0, 600, 2100])
157 P_M3_start = np.array([18000, -600, 1900])
158 P_FY1_start = np.array([17800.0, 0.0, 1800.0])
159 P_FY2_start = np.array([12000,1400,1400])
160 P_FY3_start = np.array([6000,-3000,700])
161 P_FY4_start = np.array([11000,2000,1800])
162 P_FY5_start = np.array([13000,-2000,1300])
163 TARGET_M1 = np.array([0.0, 0.0, 0.0])
164 CYLINDER_CENTER = np.array([0.0, 200.0, 0.0])
165 DIR_M1 = (TARGET_M1 - P_M1_start) / np.linalg.norm(TARGET_M1 -
    P_M1_start)
166 cylinder_surface_points = generate_cylinder_points(
167     CYLINDER_CENTER, 7.0, 10.0, NUM_CYLINDER_POINTS
168 )
169
170 global_max_ans = -1
171 global_best_params = {}
172
173
174 def plot_heatmap(alpha_range, v_range, results_grid):
175     print("\n" + "*" * 30)

```

```

176     print("正在生成最终结果的热力图...")
177     plt.figure(figsize=(12, 9))
178     im = plt.imshow(results_grid.T, origin='lower', aspect='
auto',
179                     extent=[alpha_range.min(), alpha_range.max
(), v_range.min(), v_range.max()],
180                     cmap='viridis')
181     cbar = plt.colorbar()
182     cbar.set_label('Max ANS (s)')
183     plt.title('Max ANS Heatmap for Angle vs. Speed', fontsize
=16)
184     plt.xlabel('Angle (degrees)', fontsize=12)
185     plt.ylabel('Speed (V_FY1)', fontsize=12)
186     plt.show()
187
188
189 def search():
190     global global_max_ans, global_best_params
191     FY = P_FY5_start
192
193     # --- 定义搜索范围 ---
194     v_min, v_max = 70, 140
195     alpha_min_deg, alpha_max_deg = 20, 150
196     x_min, x_max = 1000, 3500
197
198     alpha_range_deg = np.arange(alpha_min_deg, alpha_max_deg +
ANGLE_INTERVAL, ANGLE_INTERVAL)
199     v_range = np.arange(v_min, v_max + V_INTERVAL, V_INTERVAL)
200     x_range = np.arange(x_min, x_max + X_INTERVAL, X_INTERVAL)
201
202     results_grid = np.full((len(alpha_range_deg), len(v_range)
), -1.0)
203
204     for i, alpha in enumerate(tqdm(alpha_range_deg, desc="
Angle Progress")):

```

```

205     alpha_rad = np.radians(alpha)
206     for j, V in enumerate(v_range):
207         if V == 0: continue
208
209         best_x_for_pair1 = -1
210         best_x_for_pair2 = -1
211         best_x_for_pair3 = -1
212         max_ans_for_pair1 = -1
213         max_ans_for_pair2 = -1
214         max_ans_for_pair3 = -1
215
216         for x in x_range:
217
218             y1 = func(x, V, alpha_rad, P_M1_start[0],
P_M1_start[1], P_M1_start[2], FY[0], FY[1], FY[2])
219             y2 = func(x, V, alpha_rad, P_M2_start[0],
P_M2_start[1], P_M2_start[2], FY[0], FY[1], FY[2])
220             y3 = func(x, V, alpha_rad, P_M3_start[0],
P_M3_start[1], P_M3_start[2], FY[0], FY[1], FY[2])
221             T = x/V
222             y = 9.8*T*T/2
223             y1,y2,y3 = min(y1,y),min(y2,y),min(y3,y)
224
225             if (y1 is None or y1 <= 0) and (y2 is None or
y2 <= 0) and (y3 is None or y3 <= 0):
226                 continue
227
228
229             if y1 >= FY[2] and y2 >= FY[2] and y3 >= FY
[2]:
230                 continue
231
232             if y1 < 0 or y1 is None:
233                 t21 = 0
234             else:

```

```

235         t21 = math.sqrt(2 * y1 / abs(A_G_Z))
236     # 释放前飞行时间 t1
237     t11 = T - t21
238     if y2 < 0 or y2 is None:
239         t22 = 0
240     else:
241         t22 = math.sqrt(2 * y2 / abs(A_G_Z))
242     # 释放前飞行时间 t1
243     t12 = T - t22
244     if y3 < 0 or y3 is None:
245         t23 = 0
246     else:
247         t23 = math.sqrt(2 * y3 / abs(A_G_Z))
248
249     t13 = T - t23
250
251     # 如果t1为负，说明无人机到达该位置前，干扰弹就
    已落地，物理上不可能
252     if t11 < 0 and t12 < 0 and t13 < 0:
253         continue
254
255     # --- 4. 运行模拟并更新最优解 ---
256     current_ans1 = calculate_ans_for_phase3(t11,
    t21, 20, alpha, V, P_M1_start, FY)
257     current_ans2 = calculate_ans_for_phase3(t12,
    t22, 20, alpha, V, P_M2_start, FY)
258     current_ans3 = calculate_ans_for_phase3(t13,
    t23, 20, alpha, V, P_M3_start, FY)
259     #print(t1, t2, alpha, V,current_ans)
260     if current_ans1 > max_ans_for_pair1:
261         max_ans_for_pair1 = current_ans1
262         best_x_for_pair1 = x
263     if current_ans2 > max_ans_for_pair2:
264         max_ans_for_pair2 = current_ans2
265         best_x_for_pair2 = x

```



```

266         if current_ans3 > max_ans_for_pair3:
267             max_ans_for_pair3 = current_ans3
268             best_x_for_pair3 = x
269
270         if max_ans_for_pair1 > -1 and max_ans_for_pair2 >
-1 and max_ans_for_pair3 > -1:
271             results_grid[i, j] = max_ans_for_pair1 +
max_ans_for_pair2 + max_ans_for_pair3
272             print(
273                 f"Angle={alpha:.2f}°, V={V:.1f} | Found
Best ANS1={max_ans_for_pair1:.3f} ANS2={max_ans_for_pair2:.3
f} ANS3={max_ans_for_pair3:.3f} at x1={best_x_for_pair1:.1f}
m x2={best_x_for_pair2:.1f}m x3={best_x_for_pair3:.1f}m")
274
275             if max_ans_for_pair1 + max_ans_for_pair2 +
max_ans_for_pair3 > global_max_ans:
276                 global_max_ans = max_ans_for_pair1 +
max_ans_for_pair2 + max_ans_for_pair3
277                 global_best_params = {
278                     'alpha_deg': alpha,
279                     'V_FY1': V,
280                     'x1': best_x_for_pair1,
281                     'x2': best_x_for_pair2,
282                     'x3': best_x_for_pair3,
283                     'ANS': max_ans_for_pair1 +
max_ans_for_pair2 + max_ans_for_pair3
284                 }
285
286         print("\n" + "=" * 50)
287         print("SEARCH COMPLETED!")
288         if global_max_ans > -1:
289             print(f"Global Best Result:")
290             print(f"    - Max ANS: {global_best_params['ANS']:.4f} s
")
291             print(f"    - Angle: {global_best_params['alpha_deg']:.2

```

```

f} degrees")
292     print(f"    - Speed: {global_best_params['V_FY1']:.1f} m
/s")
293     print(f"    - Horizontal Distance (x1): {
global_best_params['x1']:.1f} m")
294     print(f"    - Horizontal Distance (x2): {
global_best_params['x2']:.1f} m")
295     print(f"    - Horizontal Distance (x3): {
global_best_params['x3']:.1f} m")
296     else:
297         print("No valid solution found in the given parameter
range.")
298     print("=" * 50)
299
300     plot_heatmap(alpha_range_deg, v_range, results_grid)
301
302
303 if __name__ == "__main__":
304     search()

```